1.40.10 Sorting: GATE CSE 2000 | Question: 17

An array contains four occurrences of 0, five occurrences of 1, and three occurrences of 2 in any order. The array is to be sorted using swap operations (elements that are swapped need to be adjacent).

a. What is the minimum number of swaps needed to sort such an array in the worst case?
b. Give an ordering of elements in the above array so that the minimum number of swaps needed to sort the array is maximum.

gatecse-2000 algorithms sorting normal descriptive

Answer key

1.40.11 Sorting: GATE CSE 2005 | Question: 39

Suppose there are $\lceil \log n \rceil$ sorted lists of $\lfloor n/\log n \rfloor$ elements each. The time complexity of producing a sorted list of all these elements is: (Hint:Use a heap data structure)

A. $O(n \log \log n)$
B. $\Theta(n \log n)$
C. $\Omega(n \log n)$
D. $\Omega\left(n^{3/2}\right)$

gatecse-2005 algorithms sorting normal

Answer key

1.40.12 Sorting: GATE CSE 2006 | Question: 14, ISRO2011-14

Which one of the following in place sorting algorithms needs the minimum number of swaps?

A. Quick sort
B. Insertion sort
C. Selection sort
D. Heap sort

gatecse-2006 algorithms sorting easy isro2011

Answer key

1.40.13 Sorting: GATE CSE 2007 | Question: 14

Which of the following sorting algorithms has the lowest worse-case complexity?

A. Merge sort
B. Bubble sort
C. Quick sort
D. Selection sort

gatecse-2007 algorithms sorting time-complexity easy

Answer key

1.40.14 Sorting: GATE CSE 2016 | Set 1 | Question: 13

The worst case running times of *Insertion sort* , *Merge sort* and *Quick sort*, respectively are:

A. $\Theta(n \log n)$, $\Theta(n \log n)$ and $\Theta(n^2)$
B. $\Theta(n^2)$, $\Theta(n^2)$ and $\Theta(n \log n)$
C. $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n \log n)$
D. $\Theta(n^2)$, $\Theta(n \log n)$ and $\Theta(n^2)$

gatecse-2016-set1 algorithms sorting easy

Answer key

1.40.15 Sorting: GATE CSE 2016 | Set 2 | Question: 13

Assume that the algorithms considered here sort the input sequences in ascending order. If the input is already in the ascending order, which of the following are **TRUE**?

I. Quicksort runs in $\Theta(n^2)$ time
II. Bubblesort runs in $\Theta(n^2)$ time
III. Mergesort runs in $\Theta(n)$ time
IV. Insertion sort runs in $\Theta(n)$ time

A. I and II only B. I and III only C. II and IV only D. I and IV only

gatecse-2016-set2 algorithms sorting time-complexity normal ambiguous

Answer key

1.40.16 Sorting: GATE CSE 2021 | Set 1 | Question: 9

Consider the following array.

23	32	45	69	72	73	89	97

Which algorithm out of the following options uses the least number of comparisons (among the array elements) to sort the above array in ascending order?

A. Selection sort
B. Mergesort
C. Insertion sort
D. Quicksort using the last element as pivot

gatecse-2021-set1 algorithms sorting one-mark

Answer key

1.40.17 Sorting: GATE CSE 2024 | Set 2 | Question: 25

Let A be an array containing integer values. The distance of A is defined as the minimum number of elements in A that must be replaced with another integer so that the resulting array is sorted in non-decreasing order. The distance of the array $[2, 5, 3, 1, 4, 2, 6]$ is _____.

gatecse-2024-set2 numerical-answers algorithms sorting one-mark

Answer key

1.40.18 Sorting: GATE CSE 2025 | Set 2 | Question: 10

Consider an unordered list of N distinct integers.

What is the minimum number of element comparisons required to find an integer in the list that is NOT the largest in the list?

A. 1 B. $N-1$ C. N D. $2N-1$

gatecse2025-set2 algorithms sorting one-mark

Answer key

1.40.19 Sorting: GATE DA 2026 | Question: 39

Consider the problem of sorting the given array in ascending order:

$$P = [1, 2, 3, 5, 4]$$

Consider two sorting algorithms Bubble Sort (BS) and Insertion Sort (IS).
Let $\mathrm{N1}$ be the total number of comparisons done by BS on the elements of P and $\mathrm{N2}$ be the total number of comparisons done by IS on the elements of P.

Which of the following options is/are correct?

A. $\mathrm{N1} = 10,\ \mathrm{N2} = 4$
B. $\mathrm{N1} > \mathrm{N2}$
C. IS on P will perform only one swap
D. Both BS and IS on P will make at least one unnecessary comparison (i.e., comparing elements that are already in correct order)

gateda-2026 algorithms sorting multiple-selects two-marks

Answer key

1.40.20 Sorting: GATE DS&AI 2024 | Question: 35

Consider the following sorting algorithms:

i. Bubble sort
ii. Insertion sort
iii. Selection sort

Which ONE among the following choices of sorting algorithms sorts the numbers in the array $[4,3,2,1,5]$ in increasing order after exactly two passes over the array?

A. (i) only B. (iii) only C. (i) and (iii) only D. (ii) and (iii) only

gate-ds-ai-2024 algorithms sorting two-marks

Answer key

1.40.21 Sorting: GATE IT 2005 | Question: 59

Let a and b be two sorted arrays containing n integers each, in non-decreasing order. Let c be a sorted array containing $2n$ integers obtained by merging the two arrays a and b. Assuming the arrays are indexed starting from 0, consider the following four statements

I. $a[i] \geq b[i] \Rightarrow c[2i] \geq a[i]$
II. $a[i] \geq b[i] \Rightarrow c[2i] \geq b[i]$
III. $a[i] \geq b[i] \Rightarrow c[2i] \leq a[i]$
IV. $a[i] \geq b[i] \Rightarrow c[2i] \leq b[i]$

Which of the following is TRUE?

A. only I and II B. only I and IV C. only II and III D. only III and IV

gateit-2005 algorithms sorting normal

Answer key

1.40.22 Sorting: GATE IT 2008 | Question: 43

If we use Radix Sort to sort n integers in the range $\left(n^{k/2}, n^k\right]$, for some $k > 0$ which is independent of n, the time taken would be?

A. $\Theta(n)$ B. $\Theta(kn)$ C. $\Theta(n \log n)$ D. $\Theta(n^2)$

gateit-2008 algorithms sorting normal

Answer key

1.41 Space Complexity (1)

1.41.1 Space Complexity: GATE CSE 2005 | Question: 81a

```
double foo(int n)
{
   int i;
   double sum;
   if(n == 0)
   {
      return 1.0;
   }
   else
   {
      sum = 0.0;
      for(i = 0; i < n; i++)
      {
         sum += foo(i);
      }
      return sum;
   }
}
```

The space complexity of the above code is?

A. $O(1)$ B. $O(n)$ C. $O(n!)$ D. n^n

gatecse-2005 algorithms recursion normal space-complexity

Answer key

1.42 Strongly Connected Components (3)

1.42.1 Strongly Connected Components: GATE CSE 2008 | Question: 7

The most efficient algorithm for finding the number of connected components in an undirected graph on n vertices and m edges has time complexity

A. $\Theta(n)$ B. $\Theta(m)$ C. $\Theta(m+n)$ D. $\Theta(mn)$

gatecse-2008 algorithms graph-algorithms time-complexity normal strongly-connected-components

Answer key

1.42.2 Strongly Connected Components: GATE CSE 2018 | Question: 43

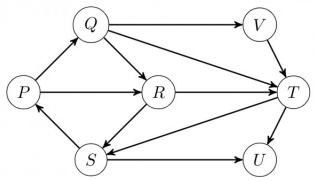
Let G be a graph with $100!$ vertices, with each vertex labelled by a distinct permutation of the numbers $1, 2, \ldots, 100$. There is an edge between vertices u and v if and only if the label of u can be obtained by swapping two adjacent numbers in the label of v. Let y denote the degree of a vertex in G, and z denote the number of connected components in G. Then, $y + 10z =$ _____.

gatecse-2018 algorithms graph-algorithms numerical-answers two-marks strongly-connected-components

Answer key

1.42.3 Strongly Connected Components: GATE IT 2006 | Question: 46

Which of the following is the correct decomposition of the directed graph given below into its strongly connected components?



A. $\{P, Q, R, S\}, \{T\}, \{U\}, \{V\}$
B. $\{P, Q, R, S, T, V\}, \{U\}$
C. $\{P, Q, S, T, V\}, \{R\}, \{U\}$
D. $\{P, Q, R, S, T, U, V\}$

gateit-2006 algorithms graph-algorithms normal strongly-connected-components

Answer key

1.43 Time Complexity (31)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (15Q) Test 4 (9Q)

1.43.1 Time Complexity: GATE CSE 1988 | Question: 6i

Given below is the sketch of a program that represents the path in a two-person game tree by the sequence of active procedure calls at any time. The program assumes that the payoffs are real number in a limited range; that the constant INF is larger than any positive payoff and its negation is smaller than any negative payoff and that there is a function “payoff” and that computes the payoff for any board that is a leaf. The type “boardtype” has been suitably declared to represent board positions. It is player-1’s move if mode = MAX and player-2’s move if mode=MIN. The type modetype = (MAX, MIN). The functions “min” and “max” find the minimum and maximum of two real numbers.

```
function search(B: boardtype; mode: modetype): real;
```

```
var
   C:boardtype; {a child of board B}
   value:real;
begin
   if B is a leaf then
      return (payoff(B))
   else
      begin
      if mode = MAX then value :=-INF
      else
         value:INF;
      for each child C of board B do
      if mode = MAX then
         value:=max (value, search (C, MIN))
      else
         value:=min(value, search(C, MAX))
      return(value)
      end
end; (search)
```

Comment on the working principle of the above program. Suggest a possible mechanism for reducing the amount of search.

gate1988 normal descriptive algorithms time-complexity

Answer key

1.43.2 Time Complexity: GATE CSE 1989 | Question: 2-iii

Match the pairs in the following:

(A)	$O(\log n)$	(p)	Heapsort
(B)	$O(n)$	(q)	Depth-first search
(C)	$O(n \log n)$	(r)	Binary search
(D)	$O(n^2)$	(s)	Selection of the k^{th} smallest element in a set of n elements

gate1989 match-the-following algorithms time-complexity

Answer key

1.43.3 Time Complexity: GATE CSE 1993 | Question: 8.7

$\sum_{1 \leq k \leq n} O(n)$, where $O(n)$ stands for order n is:

A. $O(n)$ B. $O(n^2)$ C. $O(n^3)$ D. $O(3n^2)$

E. $O(1.5n^2)$

gate1993 algorithms time-complexity easy

Answer key

1.43.4 Time Complexity: GATE CSE 1999 | Question: 1.13

Suppose we want to arrange the n numbers stored in any array such that all negative values occur before all positive ones. Minimum number of exchanges required in the worst case is

A. $n-1$ B. n C. $n+1$ D. None of the above

gate1999 algorithms time-complexity normal

Answer key

1.43.5 Time Complexity: GATE CSE 1999 | Question: 1.16

If n is a power of 2, then the minimum number of multiplications needed to compute a^n is

A. $\log_2 n$ B. $\sqrt{n}$ C. $n-1$ D. n

gate1999 algorithms time-complexity normal

Answer key

1.43.6 Time Complexity: GATE CSE 1999 | Question: 11a

Consider the following algorithms. Assume, procedure A and procedure B take $O(1)$ and $O(1/n)$ unit of time respectively. Derive the time complexity of the algorithm in O-notation.

```
algorithm what (n)
begin
   if n = 1 then call A
   else
      begin
         what (n-1);
         call B(n)
      end
end.
```

gate1999 algorithms time-complexity normal descriptive

Answer key

1.43.7 Time Complexity: GATE CSE 2000 | Question: 1.15

Let S be a sorted array of n integers. Let $T(n)$ denote the time taken for the most efficient algorithm to determined if there are two elements with sum less than 1000 in S. Which of the following statement is true?

A. $T(n)$ is $O(1)$
B. $n \leq T(n) \leq n \log_2 n$
C. $n \log_2 n \leq T(n) < \frac{n}{2}$
D. $T(n) = \left(\frac{n}{2}\right)$

gatecse-2000 easy algorithms time-complexity

Answer key

1.43.8 Time Complexity: GATE CSE 2003 | Question: 66

The cube root of a natural number n is defined as the largest natural number m such that $(m^3 \leq n)$. The complexity of computing the cube root of n (n is represented by binary notation) is

A. $O(n)$ but not $O(n^{0.5})$
B. $O(n^{0.5})$ but not $O((\log n)^k)$ for any constant $k > 0$
C. $O((\log n)^k)$ for some constant $k > 0$, but not $O((\log \log n)^m)$ for any constant $m > 0$
D. $O((\log \log n)^k)$ for some constant $k > 0.5$, but not $O((\log \log n)^{0.5})$

gatecse-2003 algorithms time-complexity normal

Answer key

1.43.9 Time Complexity: GATE CSE 2004 | Question: 39

Two matrices M_1 and M_2 are to be stored in arrays A and B respectively. Each array can be stored either in row-major or column-major order in contiguous memory locations. The time complexity of an algorithm to compute $M_1 \times M_2$ will be

A. best if A is in row-major, and B is in column-major order
B. best if both are in row-major order
C. best if both are in column-major order
D. independent of the storage scheme

gatecse-2004 algorithms time-complexity easy

Answer key

1.43.10 Time Complexity: GATE CSE 2004 | Question: 82

Let $A[1, \ldots, n]$ be an array storing a bit (1 or 0) at each location, and $f(m)$ is a function whose time complexity is $\Theta(m)$. Consider the following program fragment written in a C like language:

```
counter = 0;
for (i=1; i<=n; i++)
{
  if (a[i] == 1) counter++;
  else {f (counter); counter = 0;}
}
```

The complexity of this program fragment is

A. $\Omega(n^2)$
B. $\Omega(n \log n)$ and $O(n^2)$
C. $\Theta(n)$
D. $o(n)$

gatecse-2004 algorithms time-complexity normal

Answer key

1.43.11 Time Complexity: GATE CSE 2006 | Question: 15

Consider the following C-program fragment in which i, j and n are integer variables.

```
for( i = n, j = 0; i > 0; i /= 2, j +=i );
```

Let $val(j)$ denote the value stored in the variable j after termination of the for loop. Which one of the following is true?

A. $val(j) = \Theta(\log n)$
B. $val(j) = \Theta(\sqrt{n})$
C. $val(j) = \Theta(n)$
D. $val(j) = \Theta(n \log n)$

gatecse-2006 algorithms normal time-complexity

Answer key

1.43.12 Time Complexity: GATE CSE 2007 | Question: 15, ISRO2016-26

Consider the following segment of C-code:

```
int j, n;
j = 1;
while (j <= n)
  j = j * 2;
```

The number of comparisons made in the execution of the loop for any $n > 0$ is:

A. $\lceil \log_2 n \rceil + 1$
B. n
C. $\lceil \log_2 n \rceil$
D. $\lfloor \log_2 n \rfloor + 1$

gatecse-2007 algorithms time-complexity normal isro2016

Answer key

1.43.13 Time Complexity: GATE CSE 2007 | Question: 45

What is the time complexity of the following recursive function?

```
int  DoSomething (int n) {
  if (n <= 2)
    return 1;
  else
    return (DoSomething (floor (sqrt(n))) + n);
}
```

A. $\Theta(n^2)$
B. $\Theta(n \log_2 n)$
C. $\Theta(\log_2 n)$
D. $\Theta(\log_2 \log_2 n)$

gatecse-2007 algorithms time-complexity normal

Answer key

1.43.14 Time Complexity: GATE CSE 2007 | Question: 50

An array of n numbers is given, where n is an even number. The maximum as well as the minimum of these n numbers needs to be determined. Which of the following is **TRUE** about the number of comparisons needed?

A. At least $2n - c$ comparisons, for some constant c are needed.
B. At most $1.5n - 2$ comparisons are needed.
C. At least $n \log_2 n$ comparisons are needed
D. None of the above

gatecse-2007 algorithms time-complexity easy

Answer key

1.43.15 Time Complexity: GATE CSE 2007 | Question: 51

Consider the following C program segment:

```
int IsPrime (n)
  {
     int i, n;
     for (i=2; i<=sqrt(n);i++)
        if(n%i == 0)
           {printf("Not Prime \n"); return 0;}
     return 1;
  }
```

Let $T(n)$ denote number of times the for loop is executed by the program on input n. Which of the following is TRUE?

A. $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(\sqrt{n})$
B. $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(1)$
C. $T(n) = O(n)$ and $T(n) = \Omega(\sqrt{n})$
D. None of the above

gatecse-2007 algorithms time-complexity normal

Answer key

1.43.16 Time Complexity: GATE CSE 2008 | Question: 40

The minimum number of comparisons required to determine if an integer appears more than $\frac{n}{2}$ times in a sorted array of n integers is

A. $\Theta(n)$ B. $\Theta(\log n)$ C. $\Theta(\log^* n)$ D. $\Theta(1)$

gatecse-2008 normal algorithms time-complexity

Answer key

1.43.17 Time Complexity: GATE CSE 2008 | Question: 47

We have a binary heap on n elements and wish to insert n more elements (not necessarily one after another) into this heap. The total time required for this is

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n \log n)$ D. $\Theta(n^2)$

gatecse-2008 algorithms time-complexity normal

Answer key

1.43.18 Time Complexity: GATE CSE 2008 | Question: 74

Consider the following C functions:

```
int f1 (int n)
{
   if(n == 0 || n == 1)
      return n;
   else
```

```
    return (2 * f1(n-1) + 3 * f1(n-2));
}
int f2(int n)
{
   int i;
   int X[N], Y[N], Z[N];
   X[0] = Y[0] = Z[0] = 0;
   X[1] = 1; Y[1] = 2; Z[1] = 3;
   for(i = 2; i <= n; i++){
      X[i] = Y[i-1] + Z[i-2];
      Y[i] = 2 * X[i];
      Z[i] = 3 * X[i];
   }
   return X[n];
}
```

The running time of $f1(n)$ and $f2(n)$ are

A. $\Theta(n)$ and $\Theta(n)$
B. $\Theta(2^n)$ and $\Theta(n)$
C. $\Theta(n)$ and $\Theta(2^n)$
D. $\Theta(2^n)$ and $\Theta(2^n)$

gatecse-2008 algorithms time-complexity normal

Answer key

1.43.19 Time Complexity: GATE CSE 2008 | Question: 75

Consider the following C functions:

```
int f1 (int n)
{
   if(n == 0 || n == 1)
      return n;
   else
      return (2 * f1(n-1) + 3 * f1(n-2));
}
int f2(int n)
{
   int i;
   int X[N], Y[N], Z[N];
   X[0] = Y[0] = Z[0] = 0;
   X[1] = 1; Y[1] = 2; Z[1] = 3;
   for(i = 2; i <= n; i++){
      X[i] = Y[i-1] + Z[i-2];
      Y[i] = 2 * X[i];
      Z[i] = 3 * X[i];
   }
   return X[n];
}
```

$f1(8)$ and $f2(8)$ return the values

A. 1661 and 1640
B. 59 and 59
C. 1640 and 1640
D. 1640 and 1661

gatecse-2008 normal algorithms time-complexity

Answer key

1.43.20 Time Complexity: GATE CSE 2010 | Question: 12

Two alternative packages A and B are available for processing a database having 10^k records. Package A requires $0.0001n^2$ time units and package B requires $10n\log_{10} n$ time units to process n records. What is the smallest value of k for which package B will be preferred over A?

A. 12
B. 10
C. 6
D. 5

gatecse-2010 algorithms time-complexity easy

Answer key

1.43.21 Time Complexity: GATE CSE 2014 | Set 1 | Question: 42

Consider the following pseudo code. What is the total number of multiplications to be performed?

```
D = 2
for i = 1 to n do
```

```
for j = i to n do
   for k = j + 1 to n do
      D = D * 3
```

A. Half of the product of the 3 consecutive integers.
B. One-third of the product of the 3 consecutive integers.
C. One-sixth of the product of the 3 consecutive integers.
D. None of the above.

gatecse-2014-set1 algorithms time-complexity normal

Answer key

1.43.22 Time Complexity: GATE CSE 2015 | Set 1 | Question: 40

An algorithm performs $(\log N)^{\frac{1}{2}}$ find operations , N insert operations, $(\log N)^{\frac{1}{2}}$ delete operations, and $(\log N)^{\frac{1}{2}}$ decrease-key operations on a set of data items with keys drawn from a linearly ordered set . For a delete operation, a pointer is provided to the record that must be deleted . For the decrease-key operation, a pointer is provided to the record that has its key decreased. Which one of the following data structures is the most suited for the algorithm to use, if the goal is to achieve the best total asymptotic complexity considering all the operations?

A. Unsorted array
B. Min - heap
C. Sorted array
D. Sorted doubly linked list

gatecse-2015-set1 algorithms data-structures normal time-complexity

Answer key

1.43.23 Time Complexity: GATE CSE 2015 | Set 2 | Question: 22

An unordered list contains n distinct elements. The number of comparisons to find an element in this list that is neither maximum nor minimum is

A. $\Theta(n \log n)$ B. $\Theta(n)$ C. $\Theta(\log n)$ D. $\Theta(1)$

gatecse-2015-set2 algorithms time-complexity easy

Answer key

1.43.24 Time Complexity: GATE CSE 2017 | Set 2 | Question: 03

Match the algorithms with their time complexities:

Algorithms	Time Complexity
P. Tower of Hanoi with n disks	i. $\Theta(n^2)$
Q. Binary Search given n sorted numbers	ii. $\Theta(n \log n)$
R. Heap sort given n numbers at the worst case	iii. $\Theta(2^n)$
S. Addition of two $n \times n$ matrices	iv. $\Theta(\log n)$

A. $P \rightarrow (iii)$ $Q \rightarrow (iv)$ $r \rightarrow (i)$ $S \rightarrow (ii)$
B. $P \rightarrow (iv)$ $Q \rightarrow (iii)$ $r \rightarrow (i)$ $S \rightarrow (ii)$
C. $P \rightarrow (iii)$ $Q \rightarrow (iv)$ $r \rightarrow (ii)$ $S \rightarrow (i)$
D. $P \rightarrow (iv)$ $Q \rightarrow (iii)$ $r \rightarrow (ii)$ $S \rightarrow (i)$

gatecse-2017-set2 algorithms time-complexity match-the-following easy

Answer key

1.43.25 Time Complexity: GATE CSE 2017 | Set 2 | Question: 38

Consider the following C function

```
int fun(int n) {
   int i, j;
   for(i=1; i<=n; i++) {
```

```
    for (j=1; j<n; j+=i) {
      printf("%d %d", i, j);
    }
  }
}
```

Time complexity of fun in terms of Θ notation is

A. $\Theta(n\sqrt{n})$
B. $\Theta(n^2)$
C. $\Theta(n \log n)$
D. $\Theta(n^2 \log n)$

gatecse-2017-set2 algorithms time-complexity

Answer key

1.43.26 Time Complexity: GATE CSE 2019 | Question: 37

There are n unsorted arrays: $A_1, A_2, \ldots, A_n$. Assume that n is odd.Each of $A_1, A_2, \ldots, A_n$ contains n distinct elements. There are no common elements between any two arrays. The worst-case time complexity of computing the median of the medians of $A_1, A_2, \ldots, A_n$ is

A. $O(n)$
B. $O(n \log n)$
C. $O(n^2)$
D. $\Omega(n^2 \log n)$

gatecse-2019 algorithms time-complexity two-marks

Answer key

1.43.27 Time Complexity: GATE CSE 2024 | Set 1 | Question: 7

Given an integer array of size N, we want to check if the array is sorted (in either ascending or descending order). An algorithm solves this problem by making a single pass through the array and comparing each element of the array only with its adjacent elements. The worst-case time complexity of this algorithm is

A. both $\mathrm{O}(N)$ and $\Omega(N)$
B. $\mathrm{O}(N)$ but not $\Omega(N)$
C. $\Omega(N)$ but not $\mathrm{O}(N)$
D. neither $\mathrm{O}(N)$ nor $\Omega(N)$

gatecse-2024-set1 algorithms time-complexity one-mark

Answer key

1.43.28 Time Complexity: GATE CSE 2026 | Set 1 | Question: 7

Consider the following recurrence relations:

For all $n > 1$,

$$T_1(n) = 4T_1\left(\frac{n}{2}\right) + T_2(n)$$
$$T_2(n) = 5T_2\left(\frac{n}{4}\right) + \Theta\left(\log_2 n\right)$$

Assume that for all $n \leq 1, T_1(n) = 1$ and $T_2(n) = 1$.

Which one of the following options is correct?

A. $T_1(n) = \Theta\left(n^2\right)$
B. $T_1(n) = \Theta\left(n^2 \log_2 n\right)$
C. $T_1(n) = \Theta\left(n^{\log_4 5}\right)$
D. $T_1(n) = \Theta\left(n^{\log_4 5} \log_2 n\right)$

gatecse-2026-set1 algorithms time-complexity one-mark

Answer key

1.43.29 Time Complexity: GATE CSE 2026 | Set 2 | Question: 28

Consider an array A of integers of size n. The indices of A run from 1 to n. An algorithm is to be designed to check whether A satisfies the condition given below.

$\forall i, j \in \{1, \ldots, n-1\}$ such that $i > j, (A[i+1] - A[i]) > (A[j+1] - A[j])$

Which one of the following gives the worst case time complexity of the fastest algorithm that can be designed for the problem?

A. $\Theta(n)$
B. $\Theta(\log(n))$
C. $\Theta(n \log(n))$
D. $\Theta\left(n^2\right)$

gatecse-2026-set2 algorithms time-complexity two-marks

Answer key

1.43.30 Time Complexity: GATE IT 2007 | Question: 17

Exponentiation is a heavily used operation in public key cryptography. Which of the following options is the tightest upper bound on the number of multiplications required to compute $b^n \bmod m, 0 \leq b, n \leq m$?

A. $O(\log n)$
B. $O(\sqrt{n})$
C. $O\left(\frac{n}{\log n}\right)$
D. $O(n)$

gateit-2007 algorithms time-complexity normal

Answer key

1.43.31 Time Complexity: GATE IT 2007 | Question: 81

Let $P_1, P_2, \ldots, P_n$ be n points in the xy-plane such that no three of them are collinear. For every pair of points P_i and P_j, let L_{ij} be the line passing through them. Let L_{ab} be the line with the steepest gradient among all $n(n-1)/2$ lines.

The time complexity of the best algorithm for finding P_a and P_b is

A. $\Theta(n)$
B. $\Theta(n \log n)$
C. $\Theta(n \log^2 n)$
D. $\Theta(n^2)$

gateit-2007 algorithms time-complexity normal

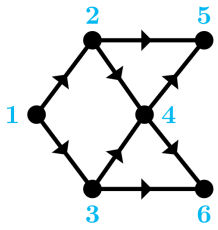
Answer key

1.44 Topological Sort (4)

✍ **Practice Test:** Test 1 (7Q)

1.44.1 Topological Sort: GATE CSE 2007 | Question: 5

Consider the DAG with $V = \{1, 2, 3, 4, 5, 6\}$ shown below.



Which of the following is not a topological ordering?

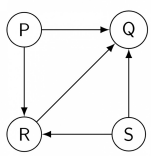
A. 1 2 3 4 5 6
B. 1 3 2 4 5 6
C. 1 3 2 4 6 5
D. 3 2 4 1 6 5

gatecse-2007 algorithms graph-algorithms topological-sort easy

Answer key

1.44.2 Topological Sort: GATE CSE 2014 | Set 1 | Question: 13

Consider the directed graph below given.



Which one of the following is **TRUE**?

A. The graph does not have any topological ordering.

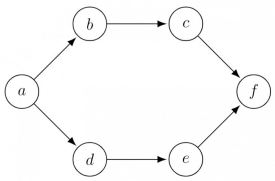
B. Both PQRS and SRQP are topological orderings.
C. Both PSRQ and SPRQ are topological orderings.
D. PSRQ is the only topological ordering.

gatecse-2014-set1 graph-algorithms easy topological-sort

Answer key

1.44.3 Topological Sort: GATE CSE 2016 | Set 1 | Question: 11

Consider the following directed graph:



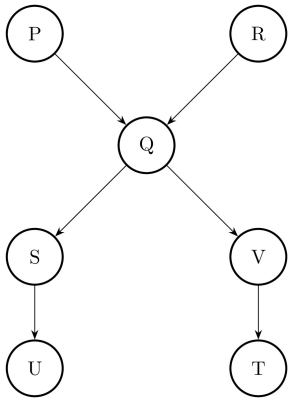
The number of different topological orderings of the vertices of the graph is _____.

gatecse-2016-set1 algorithms graph-algorithms normal numerical-answers topological-sort

Answer key

1.44.4 Topological Sort: GATE DS&AI 2024 | Question: 41

Consider the directed acyclic graph (DAG) below:



Which of the following is/are valid vertex orderings that can be obtained from a topological sort of the DAG?

A. P Q R S T U V
B. P R Q V S U T
C. P Q R S V U T
D. P R Q S V T U

gate-ds-ai-2024 algorithms topological-sort directed-acyclic-graph multiple-selects two-marks

Answer key

1.45 Tree Traversal (1)

1.45.1 Tree Traversal: GATE DA 2026 | Question: 15

You are given the following Pre-order and In-order traversals of a Binary Tree T with nodes E, F, G, P, Q, R, S.

Pre-order:	P	Q	S	E	R	F	G
In-order:	S	Q	E	P	F	R	G

Which of the following statements is/are true about the Binary Tree T?

A. Node P is the root of T
B. The Post-order traversal of T is: S
C. Node Q has only one child
D. The left subtree of node R

contains the node G

gateda-2026 algorithms tree-traversal multiple-selects one-mark

Answer key

1.46 Uniform Hashing (1)

1.46.1 Uniform Hashing: GATE CSE 2022 | Question: 6

Suppose we are given n keys, m hash table slots, and two simple uniform hash functions h_1 and h_2. Further suppose our hashing scheme uses h_1 for the odd keys and h_2 for the even keys. What is the expected number of keys in a slot?

A. $\frac{m}{n}$ B. $\frac{n}{m}$ C. $\frac{2n}{m}$ D. $\frac{n}{2m}$

gatecse-2022 algorithms hashing uniform-hashing one-mark

Answer key

Answer Keys

1.1.1	N/A	1.1.2	N/A	1.1.3	B	1.1.4	C	1.1.5	12
1.1.6	29	1.1.7	A;C	1.1.8	B	1.2.1	N/A	1.2.2	B
1.2.3	C	1.2.4	A	1.2.5	B	1.2.6	A	1.2.7	C
1.2.8	C	1.2.9	C	1.3.1	A;B	1.3.2	B	1.3.3	X
1.3.4	D	1.3.5	B	1.3.6	A	1.3.7	C	1.3.8	C
1.3.9	D	1.3.10	A	1.3.11	C	1.3.12	C	1.3.13	D
1.3.14	B	1.3.15	D	1.3.16	A	1.3.17	A;C	1.3.18	A;D
1.3.19	A	1.3.20	A	1.3.21	D	1.3.22	A	1.4.1	B
1.4.2	C	1.5.1	B;C	1.6.1	B	1.6.2	C	1.6.3	10:10
1.6.4	A	1.7.1	D	1.7.2	7 : 7	1.7.3	A;C	1.8.1	11 : 11
1.9.1	C	1.9.2	D	1.9.3	C	1.10.1	N/A	1.11.1	TBA
1.12.1	B;D	1.12.2	75:75	1.13.1	N/A	1.13.2	B	1.13.3	D
1.13.4	A	1.13.5	D	1.14.1	929 : 929	1.14.2	A	1.15.1	13
1.15.2	10:10	1.16.1	B	1.16.2	C	1.16.3	C	1.16.4	B
1.16.5	B	1.16.6	A	1.16.7	34	1.16.8	150	1.16.9	C
1.16.10	A	1.17.1	B	1.17.2	D	1.17.3	A	1.17.4	A
1.17.5	B	1.17.6	A	1.17.7	A;B	1.17.8	A	1.17.9	A
1.17.10	C	1.17.11	D	1.18.1	N/A	1.18.2	C	1.18.3	C
1.18.4	C	1.18.5	D	1.18.6	D	1.18.7	C	1.18.8	C
1.18.9	B	1.18.10	19	1.18.11	D	1.18.12	31	1.18.13	D
1.18.14	A	1.18.15	5040	1.18.16	C	1.18.17	60	1.18.18	6:6
1.18.19	A	1.18.20	D	1.18.21	D	1.18.22	D	1.18.23	B
1.19.1	A	1.19.2	B	1.19.3	D	1.19.4	A	1.19.5	16
1.20.1	B	1.20.2	N/A	1.20.3	C	1.20.4	C	1.20.5	3:3
1.20.6	D	1.21.1	C	1.21.2	D	1.22.1	2.33	1.22.2	A
1.22.3	D	1.22.4	225	1.22.5	B	1.22.6	A	1.23.1	N/A
1.23.2	N/A	1.23.3	C	1.23.4	A	1.23.5	N/A	1.23.6	C

1.23.7	C	1.23.8	N/A	1.23.9	C	1.23.10	D	1.23.11	A
1.23.12	B	1.23.13	D	1.23.14	C	1.23.15	C	1.23.16	D
1.23.17	D	1.23.18	D	1.23.19	B	1.23.20	C	1.23.21	B
1.23.22	D	1.23.23	B	1.23.24	A	1.23.25	9	1.23.26	A
1.23.27	D	1.23.28	51	1.23.29	0	1.23.30	D	1.23.31	81
1.23.32	1023 : 1023	1.23.33	15 : 15	1.23.34	D	1.23.35	C	1.23.36	C
1.23.37	D	1.23.38	C	1.24.1	A	1.24.2	D	1.25.1	B
1.25.2	A;C	1.26.1	C	1.26.2	D	1.27.1	C	1.27.2	1500
1.27.3	C	1.28.1	147.1 : 148.1	1.29.1	B	1.29.2	B	1.29.3	B
1.29.4	3:3	1.30.1	C	1.30.2	358	1.31.1	B;D;E	1.31.2	N/A
1.31.3	2	1.31.4	N/A	1.31.5	N/A	1.31.6	C	1.31.7	N/A
1.31.8	B	1.31.9	C	1.31.10	B	1.31.11	D	1.31.12	D
1.31.13	D	1.31.14	D	1.31.15	B	1.31.16	B	1.31.17	C
1.31.18	X	1.31.19	6	1.31.20	69	1.31.21	995	1.31.22	A
1.31.23	7	1.31.24	B	1.31.25	4	1.31.26	D	1.31.27	99
1.31.28	3 : 3	1.31.29	C	1.31.30	A;B;C	1.31.31	24	1.31.32	9
1.31.33	5:5	1.31.34	A;D	1.31.35	A	1.32.1	435:435	1.33.1	D
1.33.2	C	1.34.1	C	1.34.2	N/A	1.34.3	N/A	1.34.4	C
1.34.5	C	1.34.6	B	1.34.7	B	1.34.8	B	1.34.9	C
1.34.10	A	1.34.11	B	1.34.12	A	1.34.13	0.08	1.34.14	D
1.34.15	0	1.35.1	N/A	1.35.2	N/A	1.35.3	N/A	1.35.4	N/A
1.35.5	N/A	1.35.6	N/A	1.35.7	N/A	1.35.8	B	1.35.9	A
1.35.10	N/A	1.35.11	B	1.35.12	N/A	1.35.13	B	1.35.14	C
1.35.15	B	1.35.16	D	1.35.17	A	1.35.18	B	1.35.19	D
1.35.20	X	1.35.21	A	1.35.22	D	1.35.23	A	1.35.24	A
1.35.25	B	1.35.26	2.32 : 2.33	1.35.27	B	1.35.28	A	1.35.29	C
1.35.30	C	1.35.31	A	1.35.32	A	1.35.33	A;B	1.35.34	B
1.35.35	C	1.35.36	A	1.36.1	C	1.36.2	A	1.36.3	10230
1.36.4	60 : 60	1.36.5	65536 : 65536	1.37.1	N/A	1.37.2	C	1.37.3	A
1.37.4	C	1.37.5	A	1.37.6	5	1.37.7	B;C;D	1.37.8	D
1.38.1	A	1.38.2	B	1.39.1	N/A	1.39.2	B	1.39.3	D
1.39.4	A	1.39.5	B	1.39.6	C	1.39.7	A;C;D	1.39.8	C
1.40.1	N/A	1.40.2	A	1.40.3	7	1.40.4	N/A	1.40.5	D
1.40.6	N/A	1.40.7	B	1.40.8	B	1.40.9	N/A	1.40.10	N/A
1.40.11	A	1.40.12	C	1.40.13	A	1.40.14	D	1.40.15	D
1.40.16	C	1.40.17	3	1.40.18	A	1.40.19	B;C;D	1.40.20	B
1.40.21	C	1.40.22	C	1.41.1	B	1.42.1	C	1.42.2	109
1.42.3	B	1.43.1	N/A	1.43.2	N/A	1.43.3	B;C;D;E	1.43.4	D
1.43.5	A	1.43.6	N/A	1.43.7	A	1.43.8	C	1.43.9	D
1.43.10	C	1.43.11	C	1.43.12	D	1.43.13	D	1.43.14	B

1.43.15	B
1.43.20	C
1.43.25	C
1.43.30	A
1.44.4	B;D

1.43.16	B
1.43.21	C
1.43.26	C
1.43.31	B
1.45.1	A;B

1.43.17	B
1.43.22	A
1.43.27	A
1.44.1	D
1.46.1	B

1.43.18	B
1.43.23	D
1.43.28	A
1.44.2	C

1.43.19	C
1.43.24	C
1.43.29	A
1.44.3	6

Lexical analysis, Parsing, Syntax-directed translation, Runtime environments, Intermediate code generation.

Mark Distribution in Previous GATE

Year	2026 - 1	2026 - 2	2025 - 1	2025 - 2	2024 - 1	2024 - 2	2023	2022	2021 - 1	2021 - 2	Minimum
1 Mark Count	2	2	2	2	2	2	1	2	1	2	1
2 Marks Count	2	2	2	2	4	3	3	1	3	2	1
Total Marks	6	6	6	6	10	8	7	4	7	6	**4**

Subject Overview

Compiler Design is a fundamental area of Computer Science that deals with the theory and practice of building compilers – programs that translate source code written in a high-level programming language into an equivalent low-level machine-understandable form (like assembly or machine code). Understanding compiler design is crucial for a GATE CS aspirant as it provides deep insights into how programming languages work, how software interacts with hardware, and the underlying mechanisms of program execution. This subject often carries a weightage of 6-10 marks in the GATE CS exam, typically involving 2-3 questions of 2 marks and 1-2 questions of 1 mark. Questions range from theoretical concepts (phases of compilation, definitions) and conceptual understanding (optimization techniques, runtime environment) to numerical problems (parsing table construction, FIRST/FOLLOW sets, DAG generation, register allocation). A strong grasp of this subject is vital not just for the exam, but also for a holistic understanding of computer systems.

Topic-wise Key Concepts

Abstract Syntax Tree (AST)

An AST is a tree representation of the abstract syntactic structure of source code, where each node denotes a construct in the source code. It abstracts away the concrete syntax (like parentheses or semicolons) and focuses on the essential structure and meaning of the program. ASTs are crucial for semantic analysis, intermediate code generation, and various optimization techniques.

- **Key Properties:**
  - Nodes represent operators and keywords, while leaves represent operands and identifiers.
  - More compact than a parse tree, omitting unnecessary details like non-terminals that only derive other non-terminals.
  - Used as an intermediate representation for subsequent compiler phases.
- **Common Pitfalls:** Confusing AST with a parse tree. A parse tree shows every detail of the derivation; an AST shows only the essential structure.
- **Standard Problem-Solving Technique:** Given a grammar and an expression, construct its parse tree first, then derive the AST by eliminating redundant nodes and structuring it based on operator precedence and associativity.

Ambiguous Grammar

A grammar is said to be ambiguous if there exists at least one string that can be generated by the grammar in more than one leftmost derivation, or more than one rightmost derivation, or has more than one parse tree. Ambiguity is undesirable in programming language grammars because it leads to multiple interpretations of the same program statement.

- **Key Properties:**
  - Causes difficulty for parsers as they cannot uniquely determine the structure of an input string.
  - Common sources include dangling-else problem and lack of operator precedence/associativity rules.
- **Common Pitfalls:** Not being able to identify ambiguity. A common test is to find a string with two distinct parse trees.
- **Standard Problem-Solving Technique:** To resolve ambiguity, rewrite the grammar by introducing new non-terminals or by incorporating precedence and associativity rules directly into the grammar productions. For example, for arithmetic expressions, introduce non-terminals like Term and Factor.

Assembler

An assembler is a program that translates assembly language code into machine code. Assembly language uses

mnemonics for operations and symbolic names for memory locations, making it more human-readable than raw machine code. Assemblers typically perform a one-to-one translation of assembly instructions to machine instructions.

- **Key Properties:**
  - **Two-Pass Assembler:**
    1. **Pass 1:** Scans the source code to build a symbol table (mapping symbolic labels to memory addresses) and determines the length of machine instructions.
    2. **Pass 2:** Uses the symbol table to translate assembly instructions into machine code, filling in operand addresses.
  - Handles pseudo-operations (directives) like ORG, EQU, START, END, which guide the assembly process but don't translate into machine instructions.
- **Common Pitfalls:** Confusing assembler with compiler or linker. Assembler works at a lower level, translating assembly to machine code.
- **Standard Problem-Solving Technique:** Trace the assembly process, building a symbol table and calculating location counter values for each instruction and data item.

Backpatching

Backpatching is a technique used in one-pass code generation, particularly for control flow statements (like if-else, while loops) and boolean expressions. It involves generating jump instructions with initially unspecified target addresses, which are later filled in (patched) once the true target address becomes known.

- **Key Properties:**
  - Uses lists of incomplete jumps (e.g., truelist, falselist, nextlist).
  - $\text{makelist}(\text{i})$: Creates a new list containing only i (an index into the list of quadruples).
  - $\text{merge}(\text{p1}, \text{p2})$: Concatenates lists p1 and p2, returning the combined list.
  - $\text{backpatch}(\text{p}, \text{i})$: Inserts i as the target label for each of the jumps in list p.
- **Standard Problem-Solving Technique:** Trace the generation of three-address code for control flow statements, maintaining and updating truelist, falselist, and nextlist as new quadruples are generated.

Basic Blocks

A basic block is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without halt or possibility of branching except at the end. Basic blocks are fundamental units for many code optimization techniques.

- **Key Properties:**
  - No jumps into the middle of a basic block.
  - No jumps out of the middle of a basic block.
  - The first statement is called a **leader**.
- **Standard Problem-Solving Technique:**
  1. Identify **leaders**:
     - The first statement of the program is a leader.
     - Any statement that is the target of a conditional or unconditional jump is a leader.
     - Any statement immediately following a conditional or unconditional jump is a leader.
  2. For each leader, its basic block consists of the leader and all subsequent statements up to, but not including, the next leader or the end of the program.

Code Optimization

Code optimization is the phase of a compiler that attempts to improve the intermediate code or target code to make it run faster, use less memory, or consume less power, without changing the program's observable behavior. It can be machine-independent or machine-dependent.

- **Key Types/Techniques:**
  - **Machine-Independent:**
    - **Peephole Optimization:** Examines a small sliding window of instructions and replaces suboptimal sequences with better ones (e.g., redundant loads/stores, strength reduction).
    - **Common Subexpression Elimination (CSE):** Identifies and removes redundant computations of the same expression.
    - **Dead Code Elimination:** Removes code that is never executed or whose results are never used.
    - **Constant Folding:** Evaluates constant expressions at compile time (e.g., $2 * 3 + 5$ becomes 11).

        - **Loop Optimizations:** Loop invariant code motion, induction variable elimination, loop unrolling.
        - **Strength Reduction:** Replacing expensive operations with cheaper ones (e.g., $x * 2$ with $x + x$ or $x << 1$).
    - **Machine-Dependent:** Register allocation, instruction scheduling.
- **Key Property:** Optimization must preserve the semantic equivalence of the program.
- **Common Pitfalls:** Misidentifying opportunities for optimization or applying an optimization that changes program behavior.

Compilation Phases

The compilation process is typically divided into several phases, each performing a specific task in the translation of source code to target code. These phases operate sequentially, with the output of one phase serving as the input to the next.

- **Phases in Order:**
    1. **Lexical Analysis (Scanning):** Breaks source code into tokens.
    2. **Syntax Analysis (Parsing):** Checks grammar rules and builds a parse tree/AST.
    3. **Semantic Analysis:** Checks for type compatibility, undeclared variables, etc. (meaning of the program).
    4. **Intermediate Code Generation:** Creates an abstract machine-independent representation (e.g., three-address code).
    5. **Code Optimization:** Improves the intermediate or target code for efficiency.
    6. **Code Generation:** Translates intermediate code into target machine code (e.g., assembly).
- **Supporting Components:**
    - **Symbol Table Management:** Stores information about identifiers, used across multiple phases.
    - **Error Handling:** Detects and reports errors at each phase.
- **Common Pitfalls:** Confusing the order or specific responsibilities of each phase.

Compiler Tokenization (Lexical Analysis)

Compiler tokenization, or Lexical Analysis, is the first phase of a compiler where the input stream of characters is grouped into meaningful sequences called **tokens**. Each token represents a basic unit of the language, such as keywords, identifiers, operators, or literals.

- **Core Idea:** Converts a stream of characters into a stream of tokens.
- **Key Properties:**
    - Tokens are defined by **regular expressions**.
    - Lexical analyzers (scanners) are typically implemented using **Finite Automata** (NFA or DFA).
    - Handles whitespace and comments, usually discarding them.
    - **Longest Match Rule:** When multiple token patterns match a prefix of the input, the longest possible token is chosen.
    - **Precedence Rule:** If multiple patterns match the same longest prefix, the pattern listed first (or with higher precedence) is chosen.
- **Common Pitfalls:** Incorrectly applying longest match or precedence rules.
- **Standard Problem-Solving Technique:** Given a string and a set of regular expressions for tokens, identify the sequence of tokens.

Directed Acyclic Graph (DAG)

A DAG is a directed graph data structure used as an intermediate representation for basic blocks in compilers. Nodes in a DAG represent operations, and edges represent the flow of values between operations. It effectively highlights common subexpressions and the order of operations within a basic block.

- **Key Properties:**
    - Each node has a unique value. If an expression is computed multiple times, it corresponds to a single node in the DAG.
    - Leaf nodes are identifiers or constants. Interior nodes are operators.
    - Used for common subexpression elimination, dead code elimination, and reordering of operations.
- **Standard Problem-Solving Technique:**
    1. For each statement $x = y \text{ op } z$:
        - If y or z are not in the DAG, create nodes for them.
        - If op is not in the DAG with children y and z, create a node for it.
        - Add x as a label to the node representing $y \text{ op } z$.
    2. If $x = y$, make x a label for the node representing y.

Expression Evaluation

Expression evaluation is the process of computing the value of an arithmetic or logical expression according to operator precedence and associativity rules. Compilers often convert infix expressions to postfix or prefix notation for easier evaluation using a stack-based approach.

- **Key Notations:**
  - **Infix:** Operators between operands (e.g., $\mathrm{a} + \mathrm{b} * \mathrm{c}$).
  - **Postfix (Reverse Polish Notation - RPN):** Operators after operands (e.g., $\mathrm{a\ b\ c} * +$).
  - **Prefix (Polish Notation):** Operators before operands (e.g., $+ \mathrm{a} * \mathrm{b\ c}$).
- **Key Properties:**
  - **Precedence:** Determines which operator is evaluated first (e.g., multiplication before addition).
  - **Associativity:** Determines evaluation order for operators of the same precedence (e.g., left-to-right for $+$, -, $*$, $/$; right-to-left for ^).
- **Standard Problem-Solving Technique:**
  - **Infix to Postfix/Prefix:** Use a stack to manage operators based on precedence and associativity.
  - **Postfix Evaluation:** Scan from left to right. Push operands onto a stack. When an operator is encountered, pop required operands, perform operation, push result back.

First and Follow

FIRST and FOLLOW sets are crucial for constructing predictive (LL(1)) parsing tables and for resolving ambiguities in grammars. They help determine which production to apply or which token to expect next.

- **FIRST(X):** The set of terminal symbols that begin strings derived from X. If X can derive ϵ (empty string), then ϵ is in $\mathrm{FIRST(X)}$.
- **FOLLOW(A):** The set of terminal symbols that can immediately follow the non-terminal A in some sentential form. $\mathrm{FOLLOW(A)}$ never contains ϵ.
- **Important Formulas/Rules:**
  1. **FIRST for a terminal** a**:** $\mathrm{FIRST(a)} = \{\mathrm{a}\}$.
  2. **FIRST for** ϵ**:** $\mathrm{FIRST}(\epsilon) = \{\epsilon\}$.
  3. **FIRST for a non-terminal** A**:** For each production $\mathrm{A} \to \mathrm{X_1 X_2 \dots X_k}$:
     - Add $\mathrm{FIRST(X_1)} - \{\epsilon\}$ to $\mathrm{FIRST(A)}$.
     - If $\epsilon \in \mathrm{FIRST(X_i)}$ for all $\mathrm{i} = 1 \dots \mathrm{j\text{-}1}$, then add $\mathrm{FIRST(X_j)} - \{\epsilon\}$ to $\mathrm{FIRST(A)}$.
     - If $\epsilon \in \mathrm{FIRST(X_i)}$ for all $\mathrm{i} = 1 \dots \mathrm{k}$, then add ϵ to $\mathrm{FIRST(A)}$.
  4. **FOLLOW for a non-terminal** A**:**
     - Place \$ (end-marker) in $\mathrm{FOLLOW(S)}$ where S is the start symbol.
     - For each production $\mathrm{A} \to \alpha \mathrm{B} \beta$: Add $\mathrm{FIRST}(\beta) - \{\epsilon\}$ to $\mathrm{FOLLOW(B)}$.
     - For each production $\mathrm{A} \to \alpha \mathrm{B}$ or $\mathrm{A} \to \alpha \mathrm{B} \beta$ where $\epsilon \in \mathrm{FIRST}(\beta)$: Add $\mathrm{FOLLOW(A)}$ to $\mathrm{FOLLOW(B)}$.
- **Common Pitfalls:** Incorrectly handling ϵ productions, or not iterating until sets stabilize.
- **Standard Problem-Solving Technique:** Apply rules iteratively until no new terminals can be added to any set.

Grammar

A grammar is a formal system that describes the syntax of a language. It consists of a set of terminal symbols (the basic symbols of the language), non-terminal symbols (syntactic variables), a start symbol, and a set of production rules that specify how non-terminals can be replaced by sequences of terminals and non-terminals.

- **Formal Definition:** A Context-Free Grammar (CFG) is a 4-tuple $\mathrm{G} = (\mathrm{V}, \mathrm{T}, \mathrm{P}, \mathrm{S})$, where:
  - V: Finite set of non-terminal symbols.
  - T: Finite set of terminal symbols ($\mathrm{V} \cap \mathrm{T} = \emptyset$).
  - P: Finite set of production rules of the form $\mathrm{A} \to \alpha$, where $\mathrm{A} \in \mathrm{V}$ and $\alpha \in (\mathrm{V} \cup \mathrm{T})^*$.
  - \(\text

2.1 Abstract Syntax Tree (1)

2.1.1 Abstract Syntax Tree: GATE CSE 2015 | Set 2 | Question: 14

In the context of abstract-syntax-tree (AST) and control-flow-graph (CFG), which one of the following is TRUE?

A. In both AST and CFG, let node N_2 be the successor of node N_1. In the input program, the code corresponding to N_2 is present after the code corresponding to N_1
B. For any input program, neither AST nor CFG will contain a cycle
C. The maximum number of successors of a node in an AST and a CFG depends on the input program
D. Each node in AST and CFG corresponds to at most one statement in the input program

gatecse-2015-set2 compiler-design easy abstract-syntax-tree code-optimization

Answer key

2.2 Ambiguous Grammar (2)

2.2.1 Ambiguous Grammar: GATE CSE 1987 | Question: 1-xii

A context-free grammar is ambiguous if:

A. The grammar contains useless non-terminals.
B. It produces more than one parse tree for some sentence.
C. Some production has two non terminals side by side on the right-hand side.
D. None of the above.

gate1987 compiler-design parsing ambiguous-grammar

Answer key

2.2.2 Ambiguous Grammar: GATE CSE 2026 | Set 2 | Question: 19

Which of the following grammars is/are ambiguous?

A. $S \rightarrow aSb \mid \epsilon$
B. $E \rightarrow E + E | E * E | id$
C. $S \rightarrow aS | Sa | \epsilon$
D. $S \rightarrow aS \mid \epsilon$

gatecse-2026-set2 compiler-design ambiguous-grammar multiple-selects one-mark

Answer key

2.3 Assembler (9)

2.3.1 Assembler: GATE CSE 1992 | Question: 01,viii

The purpose of instruction location counter in an assembler is _____

gate1992 compiler-design assembler normal fill-in-the-blanks

Answer key

2.3.2 Assembler: GATE CSE 1992 | Question: 03,ii

Mention the pass number for each of the following activities that occur in a two pass assembler:

A. object code generation
B. literals added to literal table
C. listing printed
D. address resolution of local symbols

gate1992 compiler-design assembler easy

Answer key

2.3.3 Assembler: GATE CSE 1992 | Question: 3,i

Write short answers to the following:

i. Which of the following macros can put a macro assembler into an infinite loop?

```
.MACRO M1,X
.IF EQ,X
M1 X+1
.ENDC
.IF NE,X
.WORD X
.ENDC
.ENDM
```

```
.MACRO M2,X
.IF EQ,X
M2 X
.ENDC
.IF NE,X
.WORD X+1
.ENDC
.ENDM
```

Give an example call that does so.

gate1992 compiler-design assembler normal descriptive

Answer key

2.3.4 Assembler: GATE CSE 1993 | Question: 7.6

A simple two-pass assembler does the following in the first pass:

A. It allocates space for the literals.
B. It computes the total length of the program.
C. It builds the symbol table for the symbols and their values.
D. It generates code for all the load and store register instructions.
E. None of the above.

gate1993 compiler-design assembler easy multiple-selects

Answer key

2.3.5 Assembler: GATE CSE 1994 | Question: 17a

State whether the following statements are True or False with reasons for your answer:

Coroutine is just another name for a subroutine.

gate1994 compiler-design normal assembler true-false descriptive

Answer key

2.3.6 Assembler: GATE CSE 1994 | Question: 17b

State whether the following statements are True or False with reasons for your answer:

A two pass assembler uses its machine opcode table in the first pass of assembly.

gate1994 compiler-design normal assembler true-false descriptive

Answer key

2.3.7 Assembler: GATE CSE 1994 | Question: 18a

State whether the following statements are True or False with reasons for your answer

A subroutine cannot always be used to replace a macro in an assembly language program.

gate1994 compiler-design normal assembler true-false descriptive

Answer key

2.3.8 Assembler: GATE CSE 1994 | Question: 18b

State whether the following statements are True or False with reasons for your answer

A symbol declared as ‘external’ in an assembly language program is assigned an address outside the program by the assembler itself.

gate1994 compiler-design normal assembler true-false descriptive

Answer key

2.3.9 Assembler: GATE CSE 1996 | Question: 1.17

The pass numbers for each of the following activities

i. object code generation
ii. literals added to literal table
iii. listing printed
iv. address resolution of local symbols that occur in a two pass assembler

respectively are

A. $1,2,1,2$ B. $2,1,2,1$ C. $2,1,1,2$ D. $1,2,2,2$

gate1996 compiler-design normal assembler

Answer key

2.4 Backpatching (1)

2.4.1 Backpatching: GATE CSE 2025 | Set 2 | Question: 11

Consider the following statements about the use of backpatching in a compiler for intermediate code generation:

I. Backpatching can be used to generate code for Boolean expression in one pass.
II. Backpatching can be used to generate code for flow-of-control statements in one pass.

Which ONE of the folloeing options is CORRECT?

A. Only I is correct
B. Only II is correct
C. Both I and II are correct
D. Neither I nor II is correct

gatecse2025-set2 compiler-design backpatching intermediate-code one-mark

Answer key

2.5 Basic Blocks (2)

2.5.1 Basic Blocks: GATE CSE 2025 | Set 1 | Question: 42

Refer to the given 3-address code sequence. This code sequence is split into basic blocks. The number of basic blocks is _____. (Answer in integer)

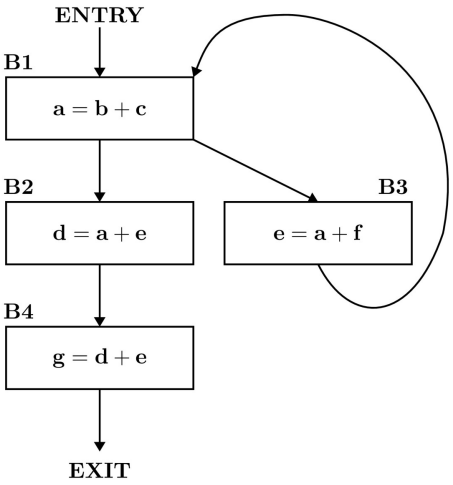
```
1001: i = 1
1002: j = 1
1003: t1 = 10*i
1004: t2 = t1+j
1005: t3 = 8*t2
1006: t4 = t3-88
1007: a[t4] = 0.0
1008: j = j+1
1009: if j <= 10 goto 1003
1010: i = i+1
1011: if i <= 10 goto 1002
1012: i = 1
1013: t5 = i-1
1014: t6 = 88*t5
1015: a[t6] = 1.0
1016: i = i+1
1017: if i <= 10 goto 1013
```

gatecse2025-set1 compiler-design three-address-code basic-blocks numerical-answers easy two-marks

Answer key

2.5.2 Basic Blocks: GATE CSE 2026 | Set 2 | Question: 35

Consider the control flow graph given below.



Which one of the following options is the set of live variables at the exit point of each basic block?

A. $B1 : \{a, b, c, e, f\}, B2 : \{d, e\}, B3 : \{b, c, e, f\}, B4 : \varnothing$
B. $B1: \varnothing$, $B2: \{ d, e \}$, $B3: \{ a, c, f \}$, $B4: \varnothing$
C. $B1 : \{a, b, c, e, f\}, B2 : \{d, e\}, B3 : \{c, e, f\}, B4 : \varnothing$
D. $B1 : \varnothing, B2 : \{d, e, f\}, B3 : \{a, b, c, e, f\}, B4 : \varnothing$

gatecse-2026-set2 compiler-design basic-blocks two-marks

Answer key

2.6 Code Optimization (8)

✍ **Practice Test:** Test 1 (13Q)

2.6.1 Code Optimization: GATE CSE 2006 | Question: 55

Consider these two functions and two statements S1 and S2 about them.

```
int work1(int *a, int i, int j)
{
    int x = a[i+2];
    a[j] = x+1;
    return a[i+2] - 3;
}
```

```
int work2(int *a, int i, int j)
{
    int t1 = i+2;
    int t2 = a[t1];
    a[j] = t2+1;
    return t2 - 3;
}
```

S1: The transformation form *work1* to *work2* is valid, i.e., for any program state and input arguments, *work2* will compute the same output and have the same effect on program state as *work1*

S2: All the transformations applied to *work1* to get *work2* will always improve the performance (i.e reduce CPU time) of *work2* compared to *work1*

A. S1 is false and S2 is false
B. S1 is false and S2 is true
C. S1 is true and S2 is false
D. S1 is true and S2 is true

gatecse-2006 compiler-design normal code-optimization

Answer key

2.6.2 Code Optimization: GATE CSE 2006 | Question: 60

Consider the following C code segment.

```
for (i = 0, i < n; i++)
{
    for (j = 0; j < n; j++)
```

```
  {
    if (i%2)
    {
      x += (4*j + 5*i);
      y += (7 + 4*j);
    }
  }
}
```

Which one of the following is false?

A. The code contains loop invariant computation
B. There is scope of common sub-expression elimination in this code
C. There is scope of strength reduction in this code
D. There is scope of dead code elimination in this code

gatecse-2006 compiler-design code-optimization

Answer key

2.6.3 Code Optimization: GATE CSE 2008 | Question: 12

Some code optimizations are carried out on the intermediate code because

A. They enhance the portability of the compiler to the target processor
B. Program analysis is more accurate on intermediate code than on machine code
C. The information from dataflow analysis cannot otherwise be used for optimization
D. The information from the front end cannot otherwise be used for optimization

gatecse-2008 normal code-optimization compiler-design

Answer key

2.6.4 Code Optimization: GATE CSE 2014 | Set 1 | Question: 17

Which one of the following is **FALSE**?

A. A basic block is a sequence of instructions where control enters the sequence at the beginning and exits at the end.
B. Available expression analysis can be used for common subexpression elimination.
C. Live variable analysis can be used for dead code elimination.
D. $x = 4 * 5 \Rightarrow x = 20$ is an example of common subexpression elimination.

gatecse-2014-set1 compiler-design code-optimization normal

Answer key

2.6.5 Code Optimization: GATE CSE 2014 | Set 3 | Question: 11

The minimum number of arithmetic operations required to evaluate the polynomial $P(X) = X^5 + 4X^3 + 6X + 5$ for a given value of X, using only one temporary variable is _____.

gatecse-2014-set3 compiler-design numerical-answers normal code-optimization

Answer key

2.6.6 Code Optimization: GATE CSE 2021 | Set 2 | Question: 30

Consider the following ANSI C code segment:

```
z=x + 3 + y->f1 + y->f2;
for (i = 0; i < 200; i = i + 2)
{
  if (z > i)
  {
    p = p + x + 3;
    q = q + y->f1;
  } else
  {
```

```
        p = p + y->f2;
        q = q + x + 3;
    }
}
```

Assume that the variable y points to a **struct** (allocated on the heap) containing two fields **f1** and **f2**, and the local variables **x, y, z, p, q,** and **i** are allotted registers. Common sub-expression elimination (CSE) optimization is applied on the code. The number of addition and the dereference operations (of the form **y->f1** or **y->f2**) in the optimized code, respectively, are:

A. 403 and 102 B. 203 and 2 C. 303 and 102 D. 303 and 2

gatecse-2021-set2 code-optimization compiler-design two-marks

Answer key

2.6.7 Code Optimization: GATE CSE 2025 | Set 1 | Question: 3

Which ONE of the following techniques used in compiler code optimization uses live variable analysis?

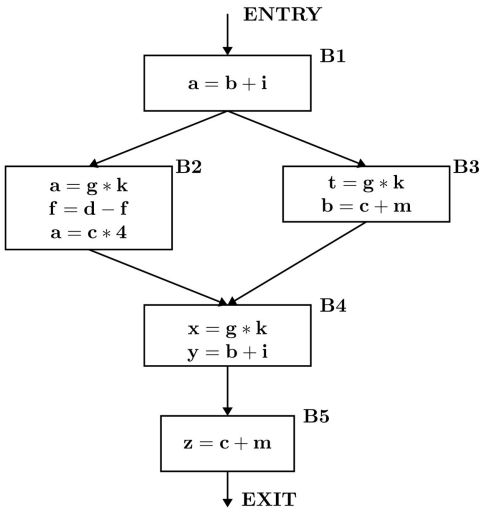
A. Run-time function call management
B. Register assignment to variables
C. Strength reduction
D. Constant folding

gatecse2025-set1 compiler-design code-optimization easy one-mark

Answer key

2.6.8 Code Optimization: GATE CSE 2026 | Set 1 | Question: 32

Consider the control flow graph shown in the figure.



Which one of the following options correctly lists the set of redundant expressions (common subexpressions) in the basic blocks B4 and B5?

Note: All the variables are integers.

A. B4: $\{b+i\}$ B5: $\{c+m\}$
B. B4: $\{g*k\}$ B5: $\{c+m\}$
C. B4: $\{g*k, b+i\}$ B5: $\{\}$
D. B4: $\{g*k\}$ B5: $\{\}$

gatecse-2026-set1 compiler-design two-marks code-optimization

Answer key

2.7 Compilation Phases (13)

Practice Test: Test 1 (6Q)

2.7.1 Compilation Phases: GATE CSE 1987 | Question: 1-xi

In a compiler the module that checks every character of the source text is called:

A. The code generator.
B. The code optimiser.
C. The lexical analyser.
D. The syntax analyser.

gate1987 compiler-design compilation-phases lexical-analysis

Answer key

2.7.2 Compilation Phases: GATE CSE 1990 | Question: 2-ix

Match the pairs in the following questions:

(a)	Lexical analysis	(p)	DAG's
(b)	Code optimization	(q)	Syntax trees
(c)	Code generation	(r)	Push down automaton
(d)	Abelian groups	(s)	Finite automaton

gate1990 match-the-following compiler-design compilation-phases

Answer key

2.7.3 Compilation Phases: GATE CSE 2005 | Question: 61

Consider line number 3 of the following C-program.

```
int main() {              /*Line 1 */
  int I, N;               /*Line 2 */
  fro (I=0, I<N, I++); /*Line 3 */
}
```

Identify the compiler's response about this line while creating the object-module:

A. No compilation error
B. Only a lexical error
C. Only syntactic errors
D. Both lexical and syntactic errors

gatecse-2005 compiler-design compilation-phases normal

Answer key

2.7.4 Compilation Phases: GATE CSE 2009 | Question: 17

Match all items in Group 1 with the correct options from those given in Group 2.

Group 1		Group 2	
P.	Regular Expression	1.	Syntax analysis
Q.	Pushdown automata	2.	Code generation
R.	Dataflow analysis	3.	Lexical analysis
S.	Register allocation	4.	Code optimization

A. P-4, Q-1, R-2, S-3
B. P-3, Q-1, R-4, S-2
C. P-3, Q-4, R-1, S-2
D. P-2, Q-1, R-4, S-3

gatecse-2009 compiler-design easy compilation-phases match-the-following

Answer key

2.7.5 Compilation Phases: GATE CSE 2015 | Set 2 | Question: 19

Match the following:

P.	Lexical analysis	1.	Graph coloring
Q.	Parsing	2.	DFA minimization
R.	Register allocation	3.	Post-order traversal
S.	Expression evaluation	4.	Production tree

A. P-2, Q-3, R-1, S-4
B. P-2, Q-1, R-4, S-3
C. P-2, Q-4, R-1, S-3
D. P-2, Q-3, R-4, S-1

gatecse-2015-set2 compiler-design normal compilation-phases match-the-following

Answer key

2.7.6 Compilation Phases: GATE CSE 2016 | Set 2 | Question: 19

Match the following:

(P)	Lexical analysis	(i)	Leftmost derivation
(Q)	Top down parsing	(ii)	Type checking
(R)	Semantic analysis	(iii)	Regular expressions
(S)	Runtime environment	(iv)	Activation records

A. P $\leftrightarrow$ i, Q $\leftrightarrow$ ii, R $\leftrightarrow$ iv, S $\leftrightarrow$ iii
B. P $\leftrightarrow$ iii, Q $\leftrightarrow$ i, R $\leftrightarrow$ ii, S $\leftrightarrow$ iv
C. P $\leftrightarrow$ ii, Q $\leftrightarrow$ iii, R $\leftrightarrow$ i, S $\leftrightarrow$ iv
D. P $\leftrightarrow$ iv, Q $\leftrightarrow$ i, R $\leftrightarrow$ ii, S $\leftrightarrow$ iii

gatecse-2016-set2 compiler-design easy match-the-following compilation-phases

Answer key

2.7.7 Compilation Phases: GATE CSE 2017 | Set 2 | Question: 05

Match the following according to input (from the left column) to the compiler phase (in the right column) that processes it:

P. Syntax tree	i. Code generator
Q. Character stream	ii. Syntax analyser
R. Intermediate representation	iii. Semantic analyser
S. Token stream	iv. Lexical analyser

A. P-ii; Q-iii; R-iv; S-i
B. P-ii; Q-i; R-iii; S-iv
C. P-iii; Q-iv; R-i; S-ii
D. P-i; Q-iv; R-ii; S-iii

gatecse-2017-set2 compiler-design match-the-following compilation-phases easy

Answer key

2.7.8 Compilation Phases: GATE CSE 2018 | Question: 8

Which one of the following statements is FALSE?

A. Context-free grammar can be used to specify both lexical and syntax rules
B. Type checking is done before parsing
C. High-level language programs can be translated to different Intermediate Representations
D. Arguments to a function can be passed using the program stack

gatecse-2018 compiler-design easy compilation-phases one-mark

Answer key

2.7.9 Compilation Phases: GATE CSE 2020 | Question: 9

Consider the following statements.

I. Symbol table is accessed only during lexical analysis and syntax analysis.
II. Compilers for programming languages that support recursion necessarily need heap storage for memory allocation in the run-time environment.

III. Errors violating the condition '*any variable must be declared before its use*' are detected during syntax analysis.

Which of the above statements is/are TRUE?

A. I only
B. I and III only
C. II only
D. None of I, II and III

gatecse-2020 compiler-design compilation-phases runtime-environment one-mark

Answer key

2.7.10 Compilation Phases: GATE CSE 2021 | Set 2 | Question: 3

Consider the following ANSI C program:

```
int main () {
  Integer x;
  return 0;
}
```

Which one of the following phases in a seven-phase C compiler will throw an error?

A. Lexical analyzer
B. Syntax analyzer
C. Semantic analyzer
D. Machine dependent optimizer

gatecse-2021-set2 compilation-phases compiler-design one-mark

Answer key

2.7.11 Compilation Phases: GATE CSE 2023 | Question: 1

Consider the following statements regarding the front-end and back-end of a compiler.

S1: The front-end includes phases that are independent of the target hardware.
S2: The back-end includes phases that are specific to the target hardware.
S3: The back-end includes phases that are specific to the programming language used in the source code.

Identify the CORRECT option.

A. Only **S1** is TRUE.
B. Only **S1** and **S2** are TRUE.
C. **S1**, **S2**, and **S3** are all TRUE.
D. Only **S1** and **S3** are TRUE.

gatecse-2023 compiler-design compilation-phases one-mark

Answer key

2.7.12 Compilation Phases: GATE CSE 2024 | Set 2 | Question: 11

Consider the following two sets:

	Set X		Set Y
P.	Lexical Analyzer	1.	Abstract Syntax Tree
Q.	Syntax Analyzer	2.	Token
R.	Intermediate Code Generator	3.	Parse Tree
S.	Code Optimizer	4.	Constant Folding

Which one of the following options is the CORRECT match from ***Set X to Set Y***?

A. $\mathrm{P}-4; \mathrm{Q}-1; \mathrm{R}-3; \mathrm{S}-2$
B. $\mathrm{P}-2; \mathrm{Q}-3; \mathrm{R}-1; \mathrm{S}-4$
C. $\mathrm{P}-2; \mathrm{Q}-1; \mathrm{R}-3; \mathrm{S}-4$
D. $\mathrm{P}-4; \mathrm{Q}-3; \mathrm{R}-2; \mathrm{S}-1$

gatecse-2024-set2 compiler-design compilation-phases match-the-following easy one-mark

Answer key

2.7.13 Compilation Phases: GATE CSE 2026 | Set 1 | Question: 17

Consider the following C statements:

```
char *str1 = "Hello;    /* Statement S1 */
char *str2 = "Hello;";  /* Statement S2 */
int *str3 = "Hello";    /* Statement S3 */
```

Which of the following options is/are correct?

A. $S1$ and $S2$ have syntactic errors
B. $S2$ has a lexical error and $S3$ has a syntactic error
C. $S1$ has a lexical error and $S3$ has a semantic error
D. $S1$ has a syntactic error and $S3$ has a semantic error

gatecse-2026-set1 compiler-design multiple-selects one-mark compilation-phases

Answer key

2.8 Compiler tokenization (1)

2.8.1 Compiler tokenization: GATE CSE 2026 | Set 2 | Question: 25

A lexical analyzer uses the following token definitions

- $letter \rightarrow [A - Za - z]$
- $digit \rightarrow [0 - 9]$
- $id \rightarrow letter(letter|digit)^*$
- $number \rightarrow digit^+$
- $ws \rightarrow (blank|tab|newline)^+$

For the string given below,

$$x1 \quad 23\,\text{mm} \quad 78 \quad \text{y} \quad 7z \quad \text{zz5} \quad 14A \quad 8H \quad \text{AaYcD}$$

the number of tokens (excluding ws) that will be produced by the lexical analyzer is____. (answer in integer)

gatecse-2026-set2 compiler-design compiler-tokenization numerical-answers one-mark

Answer key

2.9 Directed Acyclic Graph (2)

2.9.1 Directed Acyclic Graph: GATE CSE 2014 | Set 3 | Question: 34

Consider the basic block given below.

```
a = b + c
c = a + d
d = b + c
e = d - b
a = e + b
```

The minimum number of nodes and edges present in the DAG representation of the above basic block respectively are

A. 6 and 6 B. 8 and 10 C. 9 and 12 D. 4 and 4

gatecse-2014-set3 compiler-design code-optimization directed-acyclic-graph normal

Answer key

2.9.2 Directed Acyclic Graph: GATE CSE 2021 | Set 1 | Question: 50

Consider the following C code segment:

```
a = b + c;
e = a + 1;
```

```
d = b + c;
f = d + 1;
g = e + f;
```

In a compiler, this code segment is represented internally as a directed acyclic graph (DAG). The number of nodes in the DAG is _____

gatecse-2021-set1 compiler-design code-optimization directed-acyclic-graph numerical-answers two-marks

Answer key

2.10 Expression Evaluation (2)

2.10.1 Expression Evaluation: GATE CSE 2002 | Question: 2.19

To evaluate an expression without any embedded function calls

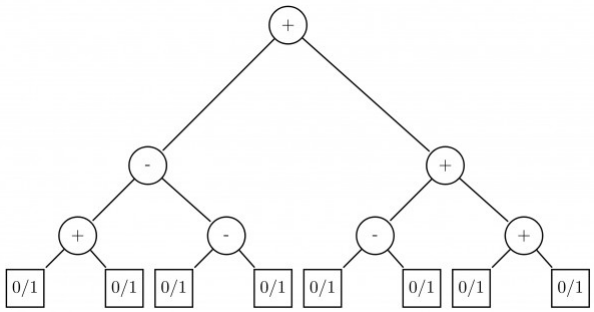
A. One stack is enough
B. Two stacks are needed
C. As many stacks as the height of the expression tree are needed
D. A Turing machine is needed in the general case

gatecse-2002 compiler-design expression-evaluation easy

Answer key

2.10.2 Expression Evaluation: GATE CSE 2014 | Set 2 | Question: 39

Consider the expression tree shown. Each leaf represents a numerical value, which can either be 0 or 1. Over all possible choices of the values at the leaves, the maximum possible value of the expression represented by the tree is ___.



gatecse-2014-set2 compiler-design normal expression-evaluation numerical-answers

Answer key

2.11 First and Follow (6)

✍ **Practice Test:** Test 1 (12Q)

2.11.1 First and Follow: GATE CSE 1992 | Question: 02,xiii

For a context free grammar, FOLLOW(A) is the set of terminals that can appear immediately to the right of non-terminal A in some "sentential" form. We define two sets LFOLLOW(A) and RFOLLOW(A) by replacing the word "sentential" by "left sentential" and "right most sentential" respectively in the definition of FOLLOW (A).

A. FOLLOW(A) and LFOLLOW(A) may be different.
B. FOLLOW(A) and RFOLLOW(A) are always the same.
C. All the three sets are identical.
D. All the three sets are different.

gate1992 parsing compiler-design normal multiple-selects first-and-follow

Answer key

2.11.2 First and Follow: GATE CSE 2012 | Question: 52

For the grammar below, a partial $LL(1)$ parsing table is also presented along with the grammar. Entries that

need to be filled are indicated as $E1, E2,$ and $E3$. ε is the empty string, \$ indicates end of input, and, | separates alternate right hand sides of productions.

- $S \rightarrow aAbB \mid bAaB \mid \varepsilon$
- $A \rightarrow S$
- $B \rightarrow S$

	a	**b**	**\$**
S	E1	E2	$S \rightarrow \varepsilon$
A	$A \rightarrow S$	$A \rightarrow S$	error
B	$B \rightarrow S$	$B \rightarrow S$	$E3$

The FIRST and FOLLOW sets for the non-terminals A and B are

A. $\text{FIRST}(A) = \{a, b, \varepsilon\} = \text{FIRST}(B)$
$\text{FOLLOW}(A) = \{a, b\}$
$\text{FOLLOW}(B) = \{a, b, \$\}$

B. $\text{FIRST}(A) = \{a, b, \$\}$
$\text{FIRST}(B) = \{a, b, \varepsilon\}$
$\text{FOLLOW}(A) = \{a, b\}$
$\text{FOLLOW}(B) = \{\$\}$

C. $\text{FIRST}(A) = \{a, b, \varepsilon\} = \text{FIRST}(B)$
$\text{FOLLOW}(A) = \{a, b\}$
$\text{FOLLOW}(B) = \varnothing$

D. $\text{FIRST}(A) = \{a, b\} = \text{FIRST}(B)$
$\text{FOLLOW}(A) = \{a, b\}$
$\text{FOLLOW}(B) = \{a, b\}$

gatecse-2012 compiler-design parsing normal first-and-follow

Answer key

2.11.3 First and Follow: GATE CSE 2017 | Set 1 | Question: 17

Consider the following grammar:

- $P \rightarrow xQRS$
- $Q \rightarrow yz \mid z$
- $R \rightarrow w \mid \varepsilon$
- $S \rightarrow y$

What is FOLLOW(Q)?

A. $\{R\}$ B. $\{w\}$ C. $\{w, y\}$ D. $\{w, \$\}$

gatecse-2017-set1 compiler-design parsing easy first-and-follow

Answer key

2.11.4 First and Follow: GATE CSE 2019 | Question: 19

Consider the grammar given below:

- $S \rightarrow Aa$
- $A \rightarrow BD$
- $B \rightarrow b \mid \epsilon$
- $D \rightarrow d \mid \epsilon$

Let a, b, d and $\$$ be indexed as follows:

a	b	d	$\$$
3	2	1	0

Compute the FOLLOW set of the non-terminal B and write the index values for the symbols in the FOLLOW set in the descending order.(For example, if the FOLLOW set is $(a, b, d, \$)$, then the answer should be 3210)

gatecse-2019 numerical-answers compiler-design parsing one-mark first-and-follow

Answer key

2.11.5 First and Follow: GATE CSE 2024 | Set 1 | Question: 28

Consider the following grammar G, with S as the start symbol. The grammar G has three incomplete productions denoted by $(1), (2)$, and (3).

$$S \rightarrow daT \mid \quad (1)$$
$$T \rightarrow aS|bT| \quad (2)$$
$$R \rightarrow (3) \mid \epsilon$$

The set of terminals is $\{a, b, c, d, f\}$. The FIRST and FOLLOW sets of the different non-terminals are as follows.

$$\text{FIRST}(S) = \{c, d, f\}, \text{FIRST}(T) = \{a, b, \epsilon\}, \text{FIRST}(R) = \{c, \epsilon\}$$
$$\text{FOLLOW}(S) = \text{FOLLOW}(T) = \{c, f, \$\}, \text{FOLLOW}(R) = \{f\}$$

Which one of the following options CORRECTLY fills in the incomplete productions?

A. (1) $S \rightarrow Rf$ (2) $T \rightarrow \epsilon$ (3) $R \rightarrow cTR$
B. (1) $S \rightarrow fR$ (2) $T \rightarrow \epsilon$ (3) $R \rightarrow cTR$
C. (1) $S \rightarrow fR$ (2) $T \rightarrow cT$ (3) $R \rightarrow cR$
D. (1) $S \rightarrow Rf$ (2) $T \rightarrow cT$ (3) $R \rightarrow cR$

gatecse-2024-set1 compiler-design first-and-follow two-marks

Answer key

2.11.6 First and Follow: GATE CSE 2025 | Set 1 | Question: 36

Which of the following statement(s) is/are TRUE while computing First and Follow during top down parsing by a compiler?

A. For a production $A \rightarrow \epsilon, \epsilon$ will be added to $\text{First}(A)$.
B. If there is any input right end marker, it will be added to First(S), where S is the start symbol.
C. For a production $A \rightarrow \epsilon, \epsilon$ will be added to Follow (A).
D. If there is any input right end marker, it will be added to $\text{Follow}(S)$, where S is the start symbol.

gatecse2025-set1 compiler-design first-and-follow parsing multiple-selects two-marks

Answer key

2.12 Grammar (47)

✍ **Practice Tests:** Test 1 (15Q) Test 2 (15Q) Test 3 (6Q)

2.12.1 Grammar: GATE CSE 1988 | Question: 10ia

Consider the following grammar:

- $S \rightarrow S$
- $S \rightarrow SS \mid a \mid \epsilon$

Construct the collection of sets of $\text{LR}\ (0)$ items for this grammar and draw its goto graph.

gate1988 compiler-design descriptive grammar parsing

Answer key

2.12.2 Grammar: GATE CSE 1988 | Question: 10ib

Consider the following grammar:

- $S \rightarrow S$
- $S \rightarrow SS \mid a \mid \epsilon$

Indicate the shift-reduce and reduce-reduce conflict (if any) in the various states of the $\mathrm{LR}(0)$ parser.

gate1988 compiler-design descriptive grammar parsing

Answer key

2.12.3 Grammar: GATE CSE 1990 | Question: 16a

Show that grammar G_1 is ambiguous using parse trees:

G_1 : $S \rightarrow$ if S then S else S

$S \rightarrow$ if S then S

gate1990 descriptive compiler-design grammar

Answer key

2.12.4 Grammar: GATE CSE 1991 | Question: 10b

Consider the following grammar for arithmetic expressions using binary operators $-$ and $/$ which are not associative

- $E \rightarrow E-T \mid T$
- $T \rightarrow T/F \mid F$
- $F \rightarrow (E) \mid id$

(E is the start symbol)

Does the grammar allow expressions with redundant parentheses as in (id/id) or in $id-(id/id)$? If so, convert the grammar into one which does not generate expressions with redundant parentheses. Do this with minimum number of changes to the given production rules and adding at most one more production rule.

gate1991 grammar compiler-design normal descriptive

Answer key

2.12.5 Grammar: GATE CSE 1991 | Question: 10c

Consider the following grammar for arithmetic expressions using binary operators $-$ and $/$ which are not associative

- $E \rightarrow E-T \mid T$
- $T \rightarrow T/F \mid F$
- $F \rightarrow (E) \mid id$

(E is the start symbol)

Does the grammar allow expressions with redundant parentheses as in (id/id) or in $id-(id/id)$? If so, convert the grammar into one which does not generate expressions with redundant parentheses. Do this with minimum number of changes to the given production rules and adding at most one more production rule.

Convert the grammar obtained above into one that is not left recursive.

gate1991 grammar compiler-design normal descriptive

Answer key

2.12.6 Grammar: GATE CSE 1992 | Question: 02,xviii

If G is a context free grammar and w is a string of length l in $L(G)$, how long is a derivation of w in G, if G is in Chomsky normal form?

A. $2l$ B. $2l+1$ C. $2l-1$ D. l

gate1992 compiler-design easy grammar

Answer key

2.12.7 Grammar: GATE CSE 1994 | Question: 1.18

Which of the following features cannot be captured by context-free grammars?

A. Syntax of if-then-else statements
B. Syntax of recursive procedures
C. Whether a variable has been declared before its use
D. Variable names of arbitrary length

gate1994 compiler-design grammar normal

Answer key

2.12.8 Grammar: GATE CSE 1994 | Question: 20

A grammar G is in Chomsky-Normal Form (CNF) if all its productions are of the form $A \rightarrow BC$ or $A \rightarrow a$, where A, B and C, are non-terminals and a is a terminal. Suppose G is a CFG in CNF and w is a string in $L(G)$ of length n, then how long is a derivation of w in G?

gate1994 compiler-design grammar normal descriptive

Answer key

2.12.9 Grammar: GATE CSE 1994 | Question: 3.5

Match the following items

(i)	Backus-Naur form	(a)	Regular expressions
(ii)	Lexical analysis	(b)	LALR(1) grammar
(iii)	YACC	(c)	LL(1) grammars
(iv)	Recursive descent parsing	(d)	General context-free grammars

gate1994 compiler-design grammar normal match-the-following

Answer key

2.12.10 Grammar: GATE CSE 1995 | Question: 1.10

Consider a grammar with the following productions

- $S \rightarrow a\alpha b \mid b\alpha c \mid aB$
- $S \rightarrow \alpha S \mid b$
- $S \rightarrow \alpha bb \mid ab$
- $S\alpha \rightarrow bdb \mid b$

The above grammar is:

A. Context free B. Regular C. Context sensitive D. $LR(k)$

gate1995 compiler-design grammar normal

Answer key

2.12.11 Grammar: GATE CSE 1995 | Question: 9

A. Translate the arithmetic expression $a^* - (b + c)$ into syntax tree.

B. A grammar is said to have cycles if it is the case that $A \overset{+}{\Rightarrow} A$

Show that no grammar that has cycles can be $\mathrm{LL}(1)$.

gate1995 compiler-design grammar normal descriptive

Answer key

2.12.12 Grammar: GATE CSE 1996 | Question: 11

Let G be a context-free grammar where $G = (\{S, A, B, C\}, \{a, b, d\}, P, S)$ with the productions in P given below.

- $S \rightarrow ABAC$
- $A \rightarrow aA \mid \varepsilon$
- $B \rightarrow bB \mid \varepsilon$
- $C \rightarrow d$

(ε denotes the null string). Transform the grammar G to an equivalent context-free grammar G' that has no ε productions and no unit productions. (A unit production is of the form $x \rightarrow y$, and x and y are non terminals).

gate1996 compiler-design grammar normal descriptive

Answer key

2.12.13 Grammar: GATE CSE 1996 | Question: 2.10

The grammar whose productions are

- $\langle \text{stmt} \rangle \rightarrow \text{ if id then } \langle \text{stmt} \rangle$
- $\langle \text{stmt} \rangle \rightarrow \text{ if id then } \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle$
- $\langle \text{stmt} \rangle \rightarrow \text{ id } := \text{ id}$

is ambiguous because

(a) the sentence

```
if a then if b then c:= d
```

has more than two parse trees

(b) the left most and right most derivations of the sentence

```
if a then if b then c:= d
```

give rise to different parse trees

(c) the sentence

```
if a then if b then c:= d else c:= f
```

has more than two parse trees

(d) the sentence

```
if a then if b then c:= d else c:= f
```

has two parse trees

gate1996 compiler-design grammar normal

Answer key

2.12.14 Grammar: GATE CSE 1997 | Question: 11

Consider the grammar

- $S \rightarrow bSe$
- $S \rightarrow PQR$

- $P \rightarrow bPc$
- $P \rightarrow \varepsilon$
- $Q \rightarrow cQd$
- $Q \rightarrow \varepsilon$
- $R \rightarrow dRe$
- $R \rightarrow \varepsilon$

where S, P, Q, R are non-terminal symbols with S being the start symbol; b, c, d, e are terminal symbols and 'ε' is the empty string. This grammar generates strings of the form b^i, c^j, d^k, e^m for some $i, j, k, m \geq 0$.

a. What is the condition on the values of i, j, k, m?
b. Find the smallest string that has two parse trees.

gate1997 compiler-design grammar normal theory-of-computation descriptive

Answer key

2.12.15 Grammar: GATE CSE 1998 | Question: 14

A. Let $G_1 = (N, T, P, S_1)$ be a CFG where, $N = \{S_1, A, B\}, T = \{a, b\}$ and P is given by

$S_1 \rightarrow aS_1b$	$S_1 \rightarrow aBb$
$S_1 \rightarrow aAb$	$B \rightarrow Bb$
$A \rightarrow aA$	$B \rightarrow b$
$A \rightarrow a$	

What is $L(G_1)$?

B. Use the grammar in Part(a) to give a CFG for $L_2 = \{a^ib^ja^kb^l \mid i, j, k, l \geq 1, i = j \text{ or } k = l\}$ by adding not more than 5 production rules.

C. Is L_2 inherently ambiguous?

gate1998 compiler-design grammar descriptive

Answer key

2.12.16 Grammar: GATE CSE 1998 | Question: 6b

Consider the grammar

- S $\rightarrow Aa \mid b$
- A $\rightarrow Ac \mid Sd \mid \epsilon$

Construct an equivalent grammar with no left recursion and with minimum number of production rules.

gate1998 compiler-design grammar descriptive

Answer key

2.12.17 Grammar: GATE CSE 1999 | Question: 2.15

A grammar that is both left and right recursive for a non-terminal, is

A. Ambiguous
B. Unambiguous
C. Information is not sufficient to decide whether it is ambiguous or unambiguous
D. None of the above

gate1999 compiler-design grammar normal

Answer key

2.12.18 Grammar: GATE CSE 2001 | Question: 1.18

Which of the following statements is false?

A. An unambiguous grammar has same leftmost and rightmost derivation
B. An LL(1) parser is a top-down parser
C. LALR is more powerful than SLR
D. An ambiguous grammar can never be LR(k) for any k

gatecse-2001 compiler-design grammar normal

Answer key

2.12.19 Grammar: GATE CSE 2001 | Question: 18

A. Remove left-recursion from the following grammar: $S \rightarrow Sa \mid Sb \mid a \mid b$
B. Consider the following grammar:

$S \rightarrow aSbS \mid bSaS \mid \epsilon$

Construct all possible parse trees for the string abab. Is the grammar ambiguous?

gatecse-2001 compiler-design grammar descriptive

Answer key

2.12.20 Grammar: GATE CSE 2003 | Question: 56

Consider the grammar shown below

- $S \rightarrow iEtSS' \mid a$
- $S' \rightarrow eS \mid \epsilon$
- $E \rightarrow b$

In the predictive parse table, M, of this grammar, the entries $M[S', e]$ and $M[S', \$]$ respectively are

A. $\{S' \rightarrow eS\}$ and $\{S' \rightarrow \epsilon\}$
B. $\{S' \rightarrow eS\}$ and $\{\}$
C. $\{S' \rightarrow \epsilon\}$ and $\{S' \rightarrow \epsilon\}$
D. $\{S' \rightarrow eS, S' \rightarrow \varepsilon\}$ and $\{S' \rightarrow \epsilon\}$

gatecse-2003 compiler-design grammar normal parsing

Answer key

2.12.21 Grammar: GATE CSE 2003 | Question: 57

Consider the grammar shown below.

- $S \rightarrow C\,C$
- $C \rightarrow c\,C \mid d$

This grammar is

A. LL(1)
B. SLR(1) but not LL(1)
C. LALR(1) but not SLR(1)
D. LR(I) but not LALR(1)

gatecse-2003 compiler-design grammar parsing normal

Answer key

2.12.22 Grammar: GATE CSE 2003 | Question: 58

Consider the translation scheme shown below.

- $S \rightarrow T\ R$
- $R \rightarrow +T\{\text{print}(\text{`}+\text{'});\}R \mid \varepsilon$
- $T \rightarrow$ **num** $\{\text{print}(\textbf{num}.\text{val});\}$

Here **num** is a token that represents an integer and **num**.val represents the corresponding integer value. For an input string '$9+5+2$', this translation scheme will print

A. $9+5+2$ B. $9\ 5+2+$ C. $9\ 5\ 2++$ D. $++9\ 5\ 2$

gatecse-2003 compiler-design grammar normal

Answer key

2.12.23 Grammar: GATE CSE 2004 | Question: 8

Which of the following grammar rules violate the requirements of an operator grammar? P, Q, R are nonterminals, and r, s, t are terminals.

I. $P \rightarrow QR$
II. $P \rightarrow QsR$
III. $P \rightarrow \varepsilon$
IV. $P \rightarrow QtRr$

A. (I) only B. (I) and (III) only C. (II) and (III) only D. (III) and (IV) only

gatecse-2004 compiler-design grammar normal

Answer key

2.12.24 Grammar: GATE CSE 2004 | Question: 88

Consider the following grammar G:

$S \rightarrow bS \mid aA \mid b$

$A \rightarrow bA \mid aB$

$B \rightarrow bB \mid aS \mid a$

Let $N_a(w)$ and $N_b(w)$ denote the number of a's and b's in a string ω respectively.

The language $L(G)$ over $\{a,b\}^+$ generated by G is

A. $\{w \mid N_a(w) > 3N_b(w)\}$
B. $\{w \mid N_b(w) > 3N_a(w)\}$
C. $\{w \mid N_a(w) = 3k, k \in \{0,1,2,\ldots\}\}$
D. $\{w \mid N_b(w) = 3k, k \in \{0,1,2,\ldots\}\}$

gatecse-2004 compiler-design grammar normal

Answer key

2.12.25 Grammar: GATE CSE 2005 | Question: 14

The grammar $A \rightarrow AA \mid (A) \mid \epsilon$ is not suitable for predictive-parsing because the grammar is:

A. ambiguous B. left-recursive C. right-recursive D. an operator-grammar

gatecse-2005 compiler-design parsing grammar easy

Answer key

2.12.26 Grammar: GATE CSE 2005 | Question: 59

Consider the grammar:

$$E \rightarrow E + n \mid E \times n \mid n$$

For a sentence $n + n \times n$, the handles in the right-sentential form of the reduction are:

A. $n, E + n$ and $E + n \times n$
B. $n, E + n$ and $E + E \times n$
C. $n, n + n$ and $n + n \times n$
D. $n, E + n$ and $E \times n$

gatecse-2005 compiler-design grammar normal

Answer key

2.12.27 Grammar: GATE CSE 2006 | Question: 32, ISRO2016-35

Consider the following statements about the context free grammar

$$G = \{S \rightarrow SS, S \rightarrow ab, S \rightarrow ba, S \rightarrow \epsilon\}$$

I. G is ambiguous
II. G produces all strings with equal number of a's and b's
III. G can be accepted by a deterministic PDA.

Which combination below expresses all the true statements about G?

A. I only
B. I and III only
C. II and III only
D. I, II and III

gatecse-2006 compiler-design grammar normal isro2016

Answer key

2.12.28 Grammar: GATE CSE 2006 | Question: 59

Consider the following translation scheme.

- $S \rightarrow ER$
- $R \rightarrow *E\,\{\text{print}(\text{`}*\text{'});\}\,R \mid \varepsilon$
- $E \rightarrow F + E\,\{\text{print}(\text{`}+\text{'});\} \mid F$
- $F \rightarrow (S) \mid id\,\{\text{print}(id.value);\}$

Here id is a token that represents an integer and $id.value$ represents the corresponding integer value. For an input '$2 * 3 + 4$', this translation scheme prints

A. $2 * 3 + 4$
B. $2 * + 3\ 4$
C. $2\ 3 * 4 +$
D. $2\ 3\ 4 + *$

gatecse-2006 compiler-design grammar normal

Answer key

2.12.29 Grammar: GATE CSE 2006 | Question: 84

Which one of the following grammars generates the language $L = \{a^i b^j \mid i \neq j\}$?

A. $S \rightarrow AC \mid CB$
$C \rightarrow aCb \mid a \mid b$
$A \rightarrow aA \mid \varepsilon$
$B \rightarrow Bb \mid \varepsilon$

B. $S \rightarrow aS \mid Sb \mid a \mid b$

C. $S \rightarrow AC \mid CB$
$C \rightarrow aCb \mid \varepsilon$
$A \rightarrow aA \mid \varepsilon$
$B \rightarrow Bb \mid \varepsilon$

D. $S \rightarrow AC \mid CB$
$C \rightarrow aCb \mid \varepsilon$
$A \rightarrow aA \mid a$
$B \rightarrow Bb \mid b$

gatecse-2006 compiler-design grammar normal theory-of-computation

Answer key

2.12.30 Grammar: GATE CSE 2006 | Question: 85

The grammar

- $S \rightarrow AC \mid CB$
- $C \rightarrow aCb \mid \epsilon$
- $A \rightarrow aA \mid a$
- $B \rightarrow Bb \mid b$

generates the language $L = \{a^i b^j \mid i \neq j\}$. In this grammar what is the length of the derivation (number of steps starting from S) to generate the string $a^l b^m$ with $l \neq m$

A. $\max(l,m)+2$ B. $l+m+2$ C. $l+m+3$ D. $\max(l,m)+3$

gatecse-2006 compiler-design grammar normal

Answer key

2.12.31 Grammar: GATE CSE 2007 | Question: 52

Consider the grammar with non-terminals $N = \{S, C, S_1\}$, terminals $T = \{a, b, i, t, e\}$, with S as the start symbol, and the following set of rules:

$S \rightarrow iCtSS_1 \mid a$

$S_1 \rightarrow eS \mid \epsilon$

$C \rightarrow b$

The grammar is NOT LL(1) because:

A. it is left recursive
B. it is right recursive
C. it is ambiguous
D. it is not context-free

gatecse-2007 compiler-design grammar normal

Answer key

2.12.32 Grammar: GATE CSE 2007 | Question: 53

Consider the following two statements:

- P: Every regular grammar is LL(1)
- Q: Every regular set has a LR(1) grammar

Which of the following is **TRUE**?

A. Both P and Q are true
B. P is true and Q is false
C. P is false and Q is true
D. Both P and Q are false

gatecse-2007 compiler-design grammar normal

Answer key

2.12.33 Grammar: GATE CSE 2007 | Question: 78

Consider the CFG with $\{S, A, B\}$ as the non-terminal alphabet, $\{a, b\}$ as the terminal alphabet, S as the start symbol and the following set of production rules:

$S \rightarrow aB$ $\quad S \rightarrow bA$

$B \rightarrow b$ $\quad A \rightarrow a$

$B \rightarrow bS$ $\quad A \rightarrow aS$

$B \rightarrow aBB$ $\quad A \rightarrow bAA$

Which of the following strings is generated by the grammar?

A. $aaaabb$ B. $aabbbb$ C. $aabbab$ D. $abbbba$

gatecse-2007 compiler-design grammar normal

Answer key

2.12.34 Grammar: GATE CSE 2007 | Question: 79

Consider the CFG with $\{S, A, B\}$ as the non-terminal alphabet, $\{a, b\}$ as the terminal alphabet, S as the start symbol and the following set of production rules:

$S \rightarrow aB \qquad S \rightarrow bA$

$B \rightarrow b \qquad A \rightarrow a$

$B \rightarrow bS \qquad A \rightarrow aS$

$B \rightarrow aBB \qquad A \rightarrow bAA$

For the string $aabbab$, how many derivation trees are there?

A. 1 B. 2 C. 3 D. 4

gatecse-2007 compiler-design grammar normal

Answer key

2.12.35 Grammar: GATE CSE 2008 | Question: 50

Which of the following statements are true?

I. Every left-recursive grammar can be converted to a right-recursive grammar and vice-versa
II. All ϵ-productions can be removed from any context-free grammar by suitable transformations
III. The language generated by a context-free grammar all of whose productions are of the form $X \rightarrow w$ or $X \rightarrow wY$ (where, w is a string of terminals and Y is a non-terminal), is always regular
IV. The derivation trees of strings generated by a context-free grammar in Chomsky Normal Form are always binary trees

A. I, II, III and IV
B. II, III and IV only
C. I, III and IV only
D. I, II and IV only

gatecse-2008 normal compiler-design grammar

Answer key

2.12.36 Grammar: GATE CSE 2010 | Question: 38

The grammar $S \rightarrow aSa \mid bS \mid c$ is

A. LL(1) but not LR(1)
B. LR(1) but not LL(1)
C. Both LL(1) and LR(1)
D. Neither LL(1) nor LR(1)

gatecse-2010 compiler-design grammar normal

Answer key

2.12.37 Grammar: GATE CSE 2016 | Set 2 | Question: 45

Which one of the following grammars is free from left recursion?

A. $S \rightarrow AB$
$A \rightarrow Aa \mid b$
$B \rightarrow c$

B. $S \rightarrow Ab \mid Bb \mid c$
$A \rightarrow Bd \mid \epsilon$
$B \rightarrow e$

C. $S \rightarrow Aa \mid B$
$A \rightarrow Bb \mid Sc \mid \epsilon$
$B \rightarrow d$

D. $S \rightarrow Aa \mid Bb \mid c$
$A \rightarrow Bd \mid \epsilon$
$B \rightarrow Ae \mid \epsilon$

gatecse-2016-set2 compiler-design grammar easy

Answer key

2.12.38 Grammar: GATE CSE 2016 | Set 2 | Question: 46

A student wrote two context-free grammars G1 and G2 for generating a single C-like array declaration. The dimension of the array is at least one. For example,

```
int a[10] [3];
```

The grammars use D as the start symbol, and use six terminal symbols **int ; id [ ] num.**

Grammar **G1**	Grammar **G2**
$\mathrm{D} \rightarrow \mathbf{int\ L;}$	$\mathrm{D} \rightarrow \mathbf{int\ L;}$
$\mathrm{L} \rightarrow \mathbf{id\ [E}$	$\mathrm{L} \rightarrow \mathbf{id\ E}$
$\mathrm{E} \rightarrow \mathbf{num\ ]}$	$\mathrm{E} \rightarrow \mathbf{E\ [num]}$
$\mathrm{E} \rightarrow \mathbf{num\ ]\ [E}$	$\mathrm{E} \rightarrow \mathbf{[num]}$

Which of the grammars correctly generate the declaration mentioned above?

A. Both **G1** and **G2** B. Only **G1** C. Only **G2** D. Neither **G1** nor **G2**

gatecse-2016-set2 compiler-design grammar normal

Answer key

2.12.39 Grammar: GATE CSE 2017 | Set 2 | Question: 32

Consider the following expression grammar G:

- $E \rightarrow E - T \mid T$
- $T \rightarrow T + F \mid F$
- $F \rightarrow (E) \mid id$

Which of the following grammars is not left recursive, but is equivalent to G?

A. $E \rightarrow E - T \mid T$
$T \rightarrow T + F \mid F$
$F \rightarrow (E) \mid id$

B. $E \rightarrow TE'$
$E' \rightarrow -TE' \mid \epsilon$
$T \rightarrow T + F \mid F$
$F \rightarrow (E) \mid id$

C. $E \rightarrow TX$
$X \rightarrow -TX \mid \epsilon$
$T \rightarrow FY$
$Y \rightarrow +FY \mid \epsilon$
$F \rightarrow (E) \mid id$

D. $E \rightarrow TX \mid (TX)$
$X \rightarrow -TX \mid +TX \mid \epsilon$
$T \rightarrow id$

gatecse-2017-set2 grammar

Answer key

2.12.40 Grammar: GATE CSE 2019 | Question: 43

Consider the augmented grammar given below:

- $S' \rightarrow S$
- $S \rightarrow \langle L \rangle \mid id$
- $L \rightarrow L, S \mid S$

Let $I_0 = \mathrm{CLOSURE}(\{[S' \rightarrow \cdot S]\})$. The number of items in the set $\mathrm{GOTO}(I_0, \langle\ )$ is_____

gatecse-2019 numerical-answers compiler-design grammar two-marks

Answer key

2.12.41 Grammar: GATE CSE 2021 | Set 1 | Question: 31

Consider the following context-free grammar where the set of terminals is $\{a, b, c, d, f\}$.

$$\begin{aligned} \mathrm{S} &\rightarrow d\,a\,\mathrm{T} \mid \mathrm{R}\,f \\ \mathrm{T} &\rightarrow a\,\mathrm{S} \mid b\,a\,\mathrm{T} \mid \epsilon \\ \mathrm{R} &\rightarrow c\,a\,\mathrm{T}\,\mathrm{R} \mid \epsilon \end{aligned}$$

The following is a partially-filled $\mathrm{LL}(1)$ parsing table.

	a	b	c	d	f	\$
S			$\boxed{1}$	$\mathrm{S} \to d a \mathrm{T}$	$\boxed{2}$	
T	$\mathrm{T} \to a \mathrm{S}$	$\mathrm{T} \to b a \mathrm{T}$	$\boxed{3}$		$\mathrm{T} \to \varepsilon$	$\boxed{4}$
R			$\mathrm{R} \to c a \mathrm{T R}$		$\mathrm{R} \to \varepsilon$	

Which one of the following choices represents the correct combination for the numbered cells in the parsing table ("blank" denotes that the corresponding cell is empty)?

A. $\boxed{1}\ \mathrm{S} \to \mathrm{R} f$ $\boxed{2}\ \mathrm{S} \to \mathrm{R} f$ $\boxed{3}\ \mathrm{T} \to \varepsilon$ $\boxed{4}\ \mathrm{T} \to \varepsilon$
B. $\boxed{1}$ blank $\boxed{2}\ \mathrm{S} \to \mathrm{R} f$ $\boxed{3}\ \mathrm{T} \to \varepsilon$ $\boxed{4}\ \mathrm{T} \to \varepsilon$
C. $\boxed{1}\ \mathrm{S} \to \mathrm{R} f$ $\boxed{2}$ blank $\boxed{3}$ blank $\boxed{4}\ \mathrm{T} \to \varepsilon$
D. $\boxed{1}$ blank $\boxed{2}\ \mathrm{S} \to \mathrm{R} f$ $\boxed{3}$ blank $\boxed{4}$ blank

gatecse-2021-set1 compiler-design grammar two-marks

Answer key

2.12.42 Grammar: GATE CSE 2024 | Set 1 | Question: 49

Let $G = (V, \Sigma, S, P)$ be a context-free grammar in Chomsky Normal Form with $\Sigma = \{a, b, c\}$ and V containing 10 variable symbols including the start symbol S. The string $w = a^{30} b^{30} c^{30}$ is derivable from S. The number of steps (application of rules) in the derivation $S \to^* w$ is _____.

gatecse-2024-set1 numerical-answers compiler-design grammar two-marks

Answer key

2.12.43 Grammar: GATE CSE 2024 | Set 2 | Question: 42

Consider a context-free grammar G with the following 3 rules.

$$S \to aS, S \to aSbS, S \to c$$

Let $w \in L(G)$. Let $n_a(w), n_b(w), n_c(w)$ denote the number of times a, b, c occur in w, respectively. Which of the following statements is/are TRUE?

A. $n_a(w) > n_b(w)$
B. $n_a(w) > n_c(w) - 2$
C. $n_c(w) = n_b(w) + 1$
D. $n_c(w) = n_b(w) * 2$

gatecse-2024-set2 compiler-design multiple-selects grammar two-marks

Answer key

2.12.44 Grammar: GATE CSE 2025 | Set 2 | Question: 41

Consider two grammars G_1 and G_2 with the production rules given below:

$G_1 : S \to if\ E\ then\ S \mid if\ E\ then\ S\ else\ S \mid a$
$\quad E \to b$
$G_2 : S \to if\ E\ then\ S \mid M$
$\quad M \to if\ E\ then\ M\ else\ S \mid c$
$\quad E \to b$

where $if, then, else, a, b, c$ are the terminals.

Which of the following option(s) is/are CORRECT?

A. G_1 is not $LL(1)$ and G_2 is $LL(1)$
B. G_1 is $LL(1)$ and G_2 is not $LL(1)$
C. G_1 and G_2 are not $LL(1)$
D. G_1 and G_2 are ambiguous.

gatecse2025-set2 compiler-design grammar multiple-selects easy two-marks

Answer key

2.12.45 Grammar: GATE IT 2005 | Question: 83a

Consider the context-free grammar

$E \rightarrow E + E$
$E \rightarrow (E * E)$
$E \rightarrow id$

where E is the starting symbol, the set of terminals is $\{id, (, +,), *\}$, and the set of nonterminals is $\{E\}$.

Which of the following terminal strings has more than one parse tree when parsed according to the above grammar?

A. $id + id + id + id$
B. $id + (id * (id * id))$
C. $(id * (id * id)) + id$
D. $((id * id + id) * id)$

gateit-2005 compiler-design grammar parsing easy

Answer key

2.12.46 Grammar: GATE IT 2007 | Question: 9

Consider an ambiguous grammar G and its disambiguated version D. Let the language recognized by the two grammars be denoted by $L(G)$ and $L(D)$ respectively. Which one of the following is true?

A. $L(D) \subset L(G)$
B. $L(D) \supset L(G)$
C. $L(D) = L(G)$
D. $L(D)$ is empty

gateit-2007 compiler-design grammar normal

Answer key

2.12.47 Grammar: GATE IT 2008 | Question: 78

A CFG G is given with the following productions where S is the start symbol, A is a non-terminal and a and b are terminals.

- $S \rightarrow aS \mid A$
- $A \rightarrow aAb \mid bAa \mid \epsilon$

Which of the following strings is generated by the grammar above?

A. $aabbaba$
B. $aabaaba$
C. $abababb$
D. $aabbaab$

gateit-2008 parsing normal grammar

Answer key

2.13 Intermediate Code (11)

✍ **Practice Test:** Test 1 (9Q)

2.13.1 Intermediate Code: GATE CSE 1988 | Question: 2xvii

Construct a DAG for the following set of quadruples:

- E:=A+B
- F:=E-C
- G:=F*D
- H:=A+B
- I:=I-C
- J:=I+G

gate1988 descriptive compiler-design intermediate-code

Answer key

2.13.2 Intermediate Code: GATE CSE 1989 | Question: 4-v

Is the following code template for the if-then-else statement correct? if not, correct it.

- if expression then statement 1
  else statement 2

Template:

Code for expression

- (*result in $E, E > O$ indicates true *)
- Branch on $E > O$ to $L1$
- Code for statement 1
- $L1$: Code for statement 2

descriptive gate1989 compiler-design intermediate-code

Answer key

2.13.3 Intermediate Code: GATE CSE 1992 | Question: 11b

Write 3 address intermediate code (quadruples) for the following boolean expression in the sequence as it would be generated by a compiler. Partial evaluation of boolean expressions is not permitted. Assume the usual rules of precedence of the operators.

$$(a+b) > (c+d) \text{ or } a > c \text{ and } b < d$$

gate1992 compiler-design syntax-directed-translation intermediate-code descriptive

Answer key

2.13.4 Intermediate Code: GATE CSE 1994 | Question: 1.12

Generation of intermediate code based on an abstract machine model is useful in compilers because

A. it makes implementation of lexical analysis and syntax analysis easier
B. syntax-directed translations can be written for intermediate code generation
C. it enhances the portability of the front end of the compiler
D. it is not possible to generate code for real machines directly from high level language programs

gate1994 compiler-design intermediate-code easy

Answer key

2.13.5 Intermediate Code: GATE CSE 2014 | Set 2 | Question: 34

For a C program accessing $\mathbf{X[i][j][k]}$, the following intermediate code is generated by a compiler. Assume that the size of an **integer** is 32 bits and the size of a **character** is 8 bits.

```
t0 = i * 1024
t1 = j * 32
t2 = k * 4
t3 = t1 + t0
t4 = t3 + t2
t5 = X[t4]
```

Which one of the following statements about the source code for the C program is CORRECT?

A. $\mathbf{X}$ is declared as **"int** $\mathbf{X[32][32][8]}$**"**.
B. $\mathbf{X}$ is declared as **"int** $\mathbf{X[4][1024][32]}$**"**.
C. $\mathbf{X}$ is declared as **"char** $\mathbf{X[4][32][8]}$**"**.
D. $\mathbf{X}$ is declared as **"char** $\mathbf{X[32][16][2]}$**"**.

gatecse-2014-set2 compiler-design intermediate-code programming-in-c normal

Answer key

2.13.6 Intermediate Code: GATE CSE 2014 | Set 3 | Question: 17

One of the purposes of using intermediate code in compilers is to

A. make parsing and semantic analysis simpler.
B. improve error recovery and error reporting.
C. increase the chances of reusing the machine-independent code optimizer in other compilers.
D. improve the register allocation.

gatecse-2014-set3 compiler-design intermediate-code easy

Answer key

2.13.7 Intermediate Code: GATE CSE 2015 | Set 1 | Question: 8

For computer based on three-address instruction formats, each address field can be used to specify which of the following:

(S1) A memory operand

(S2) A processor register

(S3) An implied accumulator register

A. Either $S1$ or $S2$
B. Either $S2$ or $S3$
C. Only $S2$ and $S3$
D. All of $S1$, $S2$ and $S3$

gatecse-2015-set1 compiler-design intermediate-code normal

Answer key

2.13.8 Intermediate Code: GATE CSE 2015 | Set 2 | Question: 29

Consider the intermediate code given below.

```
(1) i=1
(2) j=1
(3) t1 = 5 * i
(4) t2 = t1 + j
(5) t3 = 4 * t2
(6) t4 = t3
(7) a[t4] = -1
(8) j = j + 1
(9) if j <= 5 goto (3)
(10) i = i +1
(11) if i < 5 goto (2)
```

The number of nodes and edges in control-flow-graph constructed for the above code, respectively, are

A. 5 and 7
B. 6 and 7
C. 5 and 5
D. 7 and 8

gatecse-2015-set2 compiler-design intermediate-code normal

Answer key

2.13.9 Intermediate Code: GATE CSE 2021 | Set 2 | Question: 13

In the context of compilers, which of the following is/are NOT an intermediate representation of the source program?

A. Three address code
B. Abstract Syntax Tree (AST)
C. Control Flow Graph (CFG)
D. Symbol table

gatecse-2021-set2 multiple-selects compiler-design easy intermediate-code one-mark

Answer key

2.13.10 Intermediate Code: GATE CSE 2024 | Set 1 | Question: 29

Consider the following pseudo-code.

```
L 1: t1 = -1
```

```
L 2: t2 = 0
L 3: t3 = 0
L 4: t4 = 4 * t3
L 5: t5 = 4 * t2
L 6: t6 = t5 * M
L 7: t7 = t4 + t6
L 8: t8 = a[t7]
L 9: if t8 <= max goto L11
L 10: t1 = t8
L 11: t3 = t3 + 1
L 12: if t3 < M   goto L4
L 13: t2 = t2 + 1
L 14: if t2 < N goto L3
L 15: max = t1
```

Which one of the following options CORRECTLY specifies the number of basic blocks and the number of instructions in the largest basic block, respectively?

A. 6 and 6 B. 6 and 7 C. 7 and 7 D. 7 and 6

gatecse-2024-set1 compiler-design intermediate-code two-marks

Answer key

2.13.11 Intermediate Code: GATE CSE 2024 | Set 2 | Question: 33

Consider the following expression: $x[i] = (p + r) * -s[i] + u/w$. The following sequence shows the list of triples representing the given expression, with entries missing for triples (1), (3), and (6).

(0)	$+$	p	r
(1)			
(2)	uminus	(1)	
(3)			
(4)	$/$	u	w
(5)	$+$	(3)	(4)
(6)			
(7)	$=$	(6)	(5)

Which one of the following options fills in the missing entries CORRECTLY?

A. $(1)\ =[\,]\ s\ i \quad (3)\ *\ (0)(2) \quad (6)[\,]=x\ i$
B. $(1)\ [\,]=s\ i \quad (3)\ -\ (0)(2) \quad (6)\ =[\,]\ x\ (5)$
C. $(1)\ =[\,]\ s\ i \quad (3)\ *\ (0)(2) \quad (6)[\,]=x\ (5)$
D. $(1)\ [\,]=s\ i \quad (3)\ -\ (0)(2) \quad (6)\ =[\,]\ x\ i$

gatecse-2024-set2 compiler-design intermediate-code two-marks

Answer key

2.14 LR Parser (20)

Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (7Q)

2.14.1 LR Parser: GATE CSE 1992 | Question: 02,xiv

Consider the $\mathrm{SLR}(1)$ and $\mathrm{LALR}\ (1)$ parsing tables for a context free grammar. Which of the following statement is/are true?

A. The *goto* part of both tables may be different.
B. The *shift* entries are identical in both the tables.
C. The *reduce* entries in the tables may be different.
D. The *error* entries in tables may be different

gate1992 compiler-design normal parsing multiple-selects lr-parser

Answer key

2.14.2 LR Parser: GATE CSE 1998 | Question: 1.26

Which of the following statements is true?

A. SLR parser is more powerful than LALR
B. LALR parser is more powerful than Canonical LR parser
C. Canonical LR parser is more powerful than LALR parser
D. The parsers SLR, Canonical CR, and LALR have the same power

gate1998 compiler-design parsing normal lr-parser

Answer key

2.14.3 LR Parser: GATE CSE 2003 | Question: 17

Assume that the SLR parser for a grammar G has n_1 states and the LALR parser for G has n_2 states. The relationship between n_1 and n_2 is

A. n_1 is necessarily less than n_2
B. n_1 is necessarily equal to n_2
C. n_1 is necessarily greater than n_2
D. None of the above

gatecse-2003 compiler-design parsing easy lr-parser

Answer key

2.14.4 LR Parser: GATE CSE 2005 | Question: 60

Consider the grammar:

$$S \rightarrow (S) \mid a$$

Let the number of states in SLR (1), LR(1) and LALR(1) parsers for the grammar be n_1, n_2 and n_3 respectively. The following relationship holds good:

A. $n_1 < n_2 < n_3$
B. $n_1 = n_3 < n_2$
C. $n_1 = n_2 = n_3$
D. $n_1 \geq n_3 \geq n_2$

gatecse-2005 compiler-design parsing normal lr-parser

Answer key

2.14.5 LR Parser: GATE CSE 2006 | Question: 7

Consider the following grammar

- $S \rightarrow S * E$
- $S \rightarrow E$
- $E \rightarrow F + E$
- $E \rightarrow F$
- $F \rightarrow id$

Consider the following LR(0) items corresponding to the grammar above

i. $S \rightarrow S*.E$

ii. $E \rightarrow F. + E$
iii. $E \rightarrow F + . E$

Given the items above, which two of them will appear in the same set in the canonical sets-of-items for the grammar?

A. i and ii B. ii and iii C. i and iii D. None of the above

gatecse-2006 compiler-design parsing normal lr-parser

Answer key

2.14.6 LR Parser: GATE CSE 2008 | Question: 55

An LALR(1) parser for a grammar G can have shift-reduce (S-R) conflicts if and only if

A. The SLR(1) parser for G has S-R conflicts
B. The LR(1) parser for G has S-R conflicts
C. The LR(0) parser for G has S-R conflicts
D. The LALR(1) parser for G has reduce-reduce conflicts

gatecse-2008 compiler-design parsing normal lr-parser

Answer key

2.14.7 LR Parser: GATE CSE 2013 | Question: 40

Consider the following two sets of LR(1) items of an LR(1) grammar.

$X \rightarrow c.X, c/d$	$X \rightarrow c.X, \$$
$X \rightarrow .cX, c/d$	$X \rightarrow .cX, \$$
$X \rightarrow .d, c/d$	$X \rightarrow .d, \$$

Which of the following statements related to merging of the two sets in the corresponding LALR parser is/are **FALSE**?

1. Cannot be merged since look aheads are different.
2. Can be merged but will result in S-R conflict.
3. Can be merged but will result in R-R conflict.
4. Cannot be merged since goto on c will lead to two different sets.

A. 1 only B. 2 only
C. 1 and 4 only D. $1, 2, 3$ and 4

gatecse-2013 compiler-design parsing normal lr-parser

Answer key

2.14.8 LR Parser: GATE CSE 2013 | Question: 9

What is the maximum number of reduce moves that can be taken by a bottom-up parser for a grammar with no epsilon and unit-production (i.e., of type $A \rightarrow \epsilon$ and $A \rightarrow a$) to parse a string with n tokens?

A. $n/2$ B. $n-1$ C. $2n-1$ D. 2^n

gatecse-2013 compiler-design parsing normal lr-parser

Answer key

2.14.9 LR Parser: GATE CSE 2014 | Set 1 | Question: 34

A canonical set of items is given below

- $S \rightarrow L. > R$
- $Q \rightarrow R.$